ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 10. МЕТОДЫ НА ОСНОВЕ РАССТОЯНИЙ ДЛЯ ВЫЯВЛЕНИЯ АНОМАЛИЙ В КИБЕРБЕЗОПАСНОСТИ

План лекции:

1. Общая идея методов расстояний.
2. Метод ближайших соседей (kNN).
3. Local Outlier Factor (LOF).

Методы на основе расстояний для выявления аномалий в кибербезопасности

1. Общая идея методов расстояний

Методы, основанные на расстояниях, выявляют аномалии по степени удалённости объекта от других наблюдений. Основная идея состоит в том, что нормальные события образуют в пространстве признаков относительно плотные группы, а аномальные события находятся далеко от основной массы данных или имеют существенно меньше близких соседей.

Например, обычная рабочая станция может:

- устанавливать 10–30 соединений в минуту;
- обращаться к 3–8 адресатам;
- передавать небольшой объём данных.

Сетевой сеанс, содержащий 500 соединений к 200 различным адресатам, будет значительно удалён от большинства нормальных событий. Такое поведение может соответствовать сканированию сети.

Другой пример — эксфильтрация данных. Соединение может иметь обычное количество адресатов, но аномально большой объём исходящего трафика. В многомерном пространстве признаков оно также окажется удалено от основной массы нормальных объектов.

Каждый объект представляется вектором числовых признаков:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ij}, \ldots, x_{im}),$$

где:

- $\mathbf{x}_i$ — i-е событие;
- x_{ij} — значение j-го признака у i-го события;
- m — количество признаков.

Например, сетевой сеанс может быть представлен признаками:

$$\mathbf{x}_i = (\text{число соединений}, \text{число адресатов}, \text{объём трафика}).$$

После этого между событиями вычисляется расстояние: $d(\mathbf{x}_i, \mathbf{x}_q)$.

Чем меньше расстояние, тем сильнее события похожи друг на друга. Большое расстояние означает, что их характеристики существенно различаются.

Обобщённая оценка аномальности.

Для объекта $\mathbf{x}$ строится оценка аномальности:

$$S(\mathbf{x}) = f\left[d(\mathbf{x}, \mathbf{x}_{(1)}), d(\mathbf{x}, \mathbf{x}_{(2)}), \ldots, d(\mathbf{x}, \mathbf{x}_{(q)}), \ldots, d(\mathbf{x}, \mathbf{x}_{(k)})\right],$$

где:

- $S(\mathbf{x})$ — оценка аномальности объекта;
- k — количество рассматриваемых ближайших соседей;
- $\mathbf{x}_{(q)}$ — q-й ближайший сосед объекта $\mathbf{x}$;
- $d(\mathbf{x}, \mathbf{x}_{(q)})$ — расстояние от объекта $\mathbf{x}$ до q-го соседа;
- $f(\cdot)$ — функция, преобразующая расстояния в оценку аномальности.

В зависимости от конкретного метода оценкой может быть:

- расстояние до k-го ближайшего соседа;
- среднее расстояние до k соседей;
- количество объектов внутри заданного радиуса;
- отношение локальной плотности объекта к плотности его соседей.

Чем больше значение оценки, тем более аномальным считается событие.

Итоговое решение принимается с помощью порога:

$$\hat{y}(\mathbf{x}) = \begin{cases} 1, & S(\mathbf{x}) > T, \\ 0, & S(\mathbf{x}) \leq T, \end{cases},$$

где:

- $\hat{y}(\mathbf{x})$ — предсказанная метка: 1 — аномалия; 0 — нормальное событие;
- $S(\mathbf{x})$ — оценка аномальности;
- T — порог.

На практике наиболее часто используются два метода на основе расстояний:

- метод ближайших соседей (k-Nearest Neighbors - kNN);
- локальный фактор выбросов (Local Outlier Factor - LOF).

Оба метода используют ближайших соседей, но kNN измеряет абсолютную удалённость, а LOF — относительную локальную изолированность.

Методы на основе расстояний могут применяться в двух разных режимах.

1. Обучение без учителя: Outlier Detection

Алгоритм получает набор данных, в котором предположительно содержатся и нормальные события, и небольшое количество атак. Метки событий модели неизвестны.

Задача состоит в том, чтобы найти редкие наблюдения внутри этой же выборки.

Схема:

$$\text{смешанная выборка} \rightarrow \text{оценка удалённости(плотности)} \rightarrow \text{поиск редких объе}$$

Основное предположение: аномалий значительно меньше, чем нормальных событий.

Опасность состоит в том, что несколько похожих атак могут образовать собственную плотную группу. Тогда алгоритм может принять её за один из режимов нормального поведения.

2. Обучение только на нормальных данных: Novelty Detection

Имеется историческая выборка, содержащая только подтверждённые нормальные события. На ней строится профиль нормы. Затем модель применяется к новым событиям.

Схема:

$$\text{нормальные события} \rightarrow \text{модель нормы} \rightarrow \text{оценка новых событий}.$$

Метод обнаруживает не выбросы в обучающей выборке, а новые события, не соответствующие известному профилю. Такая постановка также называется обнаружением новизны.

Главная опасность — загрязнение обучающей выборки незамеченными атаками и неполное описание нормальных режимов работы.

2. Подготовка исходных данных

Для изучения методов на основе расстояний будет использоваться набор данных, содержащий агрегированные характеристики сетевых сессий организации.

Каждая сессия содержит три числовых признака:

Признак	Описание	Возможный смысл аномалии
`connections_per_min`	Количество соединений в минуту	Сканирование, автоматизированная активность
`unique_destinations`	Количество уникальных узлов назначения	Сканирование или боковое перемещение
`outbound_mb`	Объём исходящего трафика, МБ	Возможная эксфильтрация данных

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

In [3]:

```python
# @title
rng = np.random.default_rng(42)

FEATURES = [
    "connections_per_min",
    "unique_destinations",
    "outbound_mb"
]


def make_normal_group(
    n,
    mean,
    covariance,
    traffic_type,
    dataset_part
):
    """Создаёт один режим нормального сетевого поведения."""

    values = rng.multivariate_normal(
        mean=mean,
        cov=covariance,
        size=n
    )

    # Признаки не могут быть отрицательными.
    values = np.clip(
        values,
        a_min=[0.1, 0.1, 0.1],
        a_max=None
    )

    result = pd.DataFrame(
        values,
        columns=FEATURES
    )

    result["traffic_type"] = traffic_type
    result["scenario"] = "normal"
    result["is_attack"] = 0
    result["dataset_part"] = dataset_part

    return result


# Ковариационные матрицы задают разброс и взаимосвязи
# признаков внутри каждого нормального режима.
web_covariance = np.array([
    [64, 15, 20],
    [15,  9,  5],
    [20,  5, 49]
])

api_covariance = np.array([
    [225, 5, 30],
    [  5, 2.25, 2],
    [ 30, 2, 64]
])
```

```python
backup_covariance = np.array([
    [9, 1,   5],
    [1, 0.36, 1],
    [5, 1, 625]
])

# Исторические подтверждённые нормальные данные.
reference_groups = [
    make_normal_group(
        360,
        mean=[55, 14, 25],
        covariance=web_covariance,
        traffic_type="web",
        dataset_part="reference_normal"
    ),
    make_normal_group(
        180,
        mean=[135, 6, 35],
        covariance=api_covariance,
        traffic_type="api",
        dataset_part="reference_normal"
    ),
    make_normal_group(
        60,
        mean=[18, 2, 220],
        covariance=backup_covariance,
        traffic_type="backup",
        dataset_part="reference_normal"
    )
]

# Нормальная часть мониторингового набора.
monitoring_normal_groups = [
    make_normal_group(
        144,
        mean=[55, 14, 25],
        covariance=web_covariance,
        traffic_type="web",
        dataset_part="monitoring"
    ),
    make_normal_group(
        72,
        mean=[135, 6, 35],
        covariance=api_covariance,
        traffic_type="api",
        dataset_part="monitoring"
    ),
    make_normal_group(
        24,
        mean=[18, 2, 220],
        covariance=backup_covariance,
        traffic_type="backup",
        dataset_part="monitoring"
    )
]


def make_attack_group(n, mean, std, scenario):
    """Создаёт группу искусственных атак."""
```

```
    values = rng.normal(
        loc=mean,
        scale=std,
        size=(n, len(FEATURES))
    )

    values = np.clip(values, 0.1, None)

    result = pd.DataFrame(
        values,
        columns=FEATURES
    )

    result["traffic_type"] = "unknown"
    result["scenario"] = scenario
    result["is_attack"] = 1
    result["dataset_part"] = "monitoring"

    return result


attack_groups = [
    make_attack_group(
        4,
        mean=[220, 150, 5],
        std=[8, 10, 1],
        scenario="port_scan"
    ),
    make_attack_group(
        3,
        mean=[55, 13, 650],
        std=[3, 2, 30],
        scenario="exfiltration"
    ),
    make_attack_group(
        4,
        mean=[75, 1, 3],
        std=[3, 0.2, 0.5],
        scenario="beaconing"
    ),
    make_attack_group(
        3,
        mean=[145, 45, 20],
        std=[5, 3, 3],
        scenario="lateral_movement"
    )
]

df = pd.concat(
    reference_groups
    + monitoring_normal_groups
    + attack_groups,
    ignore_index=True
)

df.insert(
    0,
    "session_id",
    [f"S{i:04d}" for i in range(len(df))]
)
```

```
df[FEATURES] = df[FEATURES].round(2)
```

In [4]:
```
df.head()
```

Out[4]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	sce
0	S0000	49.93	10.93	28.74	web	n
1	S0001	41.26	13.53	30.24	web	n
2	S0002	53.06	13.54	25.97	web	n
3	S0003	64.45	14.59	24.73	web	n
4	S0004	58.31	13.76	19.01	web	n

В набор данных добавлены служебные столбцы, которые необходимы для разделения данных с целью демонстрации различных режимов работы и для проверки качества выявления аномалий:

- `dataset_part` :
  - `dataset_part == monitoring` : мониторинговая часть данных, которая будет использоваться в режиме обучения без учителя (содержит как нормальные сессии, так и атаки);
  - `dataset_part == reference_normal` : часть данных, которая будет использоваться в режиме обучения только на нормальных данных (содержит только нормальные сессии);
- `traffic_type` : истинный режим нормального трафика:
  - `web` : обычная работа пользователей с веб-ресурсами;
  - `api` — интенсивные обращения приложений к небольшому числу API;
  - `backup` — небольшое число соединений с большим объёмом данных;
- `scenario` : название сценария для наблюдение:
  - `normal` : нормальная сессия;
  - `port_scan` : большое количество соединений с множеством узлов;
  - `exfiltration` : аномально большой объём исходящего трафика;
  - `beaconing` : периодическая связь с одним адресатом небольшими пакетами;
  - `lateral_movement` : взаимодействие с нетипично большим числом внутренних узлов;
- `is_attack` : истинная метка нормы или атаки.

Эти метки не поступают на вход алгоритмов выявления аномалий.

Разделение данных.

In [5]:
```
# Исторически нормальные данные
reference_df = df[df["dataset_part"] == "reference_normal"].copy()
```

```
# Мониторинговые данные
monitoring_df = df[df["dataset_part"] == "monitoring"].copy()
```

В режиме обучения только на нормальных данных будет использоваться историческая нормальная часть `reference_normal`. На ней модель изучает нормальное поведение. Затем на данных мониторинговой части модель проверяет новые события.

В режиме обучения без учителя используется вся неразмеченная анализируемая выборка. В данной учебной демонстрации такой выборкой является часть `monitoring`, содержащая одновременно нормальные события и неизвестные алгоритму атаки. Алгоритм обучается на всех объектах `monitoring` и ищет выбросы среди этих же объектов.

Часть `reference_normal` в этот режим не включается, поскольку она специально предназначена для отдельной постановки — обучения только по подтверждённым нормальным данным.

Структура набора данных:

```
df
├── reference_normal — 600 подтверждённых нормальных сессий
└── monitoring       — 254 новые сессии
    ├── 240 нормальных сессий
    └── 14 атак
        ├── 4 port_scan
        ├── 4 beaconing
        ├── 3 exfiltration
        └── 3 lateral_movement
```

Вспомогательные функции

Создадим две вспомогательные функции, которые будут формировать результаты работы метода.

Функция для расчёта и вывода метрик качества детекции.

In [6]:
```
# @title
from sklearn.metrics import confusion_matrix
def print_detection_result(method_name, y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()

    print(method_name)
    print(f"TP — обнаруженные атаки: {tp}")
    print(f"FP — ложные тревоги: {fp}")
    print(f"FN — пропущенные атаки: {fn}")
    print(f"TN — корректно распознанная норма: {tn}")
```

Функция визуализации результатов выявления аномалий.

In [33]:
```
# @title
def plot_knn_scores(data, score_column, threshold, title):
    """
```

```
    Строит распределение оценок kNN.

    Истинные метки используются только для учебного
    отображения результата. При эксплуатации они неизвестны.
    """
    normal_scores = data.loc[data["is_attack"] == 0, score_column]

    attack_scores = data.loc[data["is_attack"] == 1, score_column]

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(normal_scores, bins=30, alpha=0.65, color="steelblue",
            label="Нормальные события")

    ax.hist(attack_scores, bins=20, alpha=0.75, color="red",
            label="Атаки")

    ax.axvline(threshold, color="black", linestyle="--",
               linewidth=2, label=f"Порог = {threshold:.3f}")

    ax.set_title(title)
    ax.set_xlabel("Среднее расстояние до ближайших соседей")
    ax.set_ylabel("Количество событий")
    ax.grid(alpha=0.25)
    ax.legend()

    plt.tight_layout()
```

3. Метод ближайших соседей kNN

Метод k ближайших соседей (K-Nearest Neighbors, kNN) предполагает, что нормальные события образуют достаточно плотные группы. У нормального события обычно есть несколько похожих соседей, расположенных на небольшом расстоянии.

Аномальное событие отличается от большинства наблюдений, поэтому расстояние до его ближайших соседей оказывается большим.

В отличие от классификационного kNN, при обнаружении аномалий голосование соседей по известным классам не выполняется. Базовый вариант использует только расстояния до соседей.

kNN может применяться в двух основных режимах:

- **обучение без учителя** (имеется неразмеченная выборка и в ней могут одновременно находиться нормальные события и неизвестные атаки);
- **обучение только на нормальных данных** (имеется эталонная нормальная выборка и все эталонные объекты считаются нормальными. Новое мониторинговое событие сравнивается только с ними).

Алгоритм kNN для выявления аномалий

Шаг 1. Формирование пространства признаков.

Каждое событие представляется вектором:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ij}, \ldots, x_{im}),$$

где:

- $\mathbf{x}_i$ — i-е событие;
- x_{ij} — значение j-го признака у i-го события;
- m — количество признаков.

Категориальные признаки, например протокол или тип сервиса, должны быть предварительно закодированы либо использованы для разделения данных на однородные группы.

Шаг 2. Масштабирование признаков.

kNN непосредственно использует расстояния, поэтому чувствителен к масштабу признаков.

Например:

- число адресатов может находиться в диапазоне от 1 до 200;
- объём трафика — от 0,1 до 1000 Мбайт;
- число соединений — от 1 до 500.

Без масштабирования признак с наибольшими числовыми значениями будет доминировать при вычислении расстояния.

Для стандартизации используется формула:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

где:

- x_{ij} — исходное значение j-го признака у i-го события;
- z_{ij}— стандартизированное значение;
- μ_j — среднее значение j-го признака в обучающей выборке;
- σ_j — стандартное отклонение j-го признака в обучающей выборке.

Различие режимов обучения.

При обучении без учителя параметры масштабирования вычисляются по всей неразмеченной выборке. Недостаток заключается в том, что атаки также влияют на средние значения и стандартные отклонения.

При обучении только на нормальных данных параметры масштабирования вычисляются исключительно по нормальной обучающей выборке. Для новых событий используются уже рассчитанные параметры. Пересчитывать масштабирование по мониторинговой выборке нельзя.

Шаг 3. Выбор количества соседей.

Параметр k определяет, сколько ближайших объектов участвует в расчёте оценки аномальности:

$$1 \leq k < n,$$

где n — количество объектов в обучающей выборке.

Малое значение k делает метод чувствительным к отдельным объектам и шуму. Два расположенных рядом атакующих события могут сделать друг друга неаномальными.

Большое значение k сглаживает локальную структуру данных. Редкий, но легитимный режим работы может быть ошибочно признан аномальным.

Не существует формализованного способа оценивания величины k. На практике используются экспериментальные и эвристические подходы.

Наиболее распрстраненным экспериментальным подходом является валидация на «чистой норме» с порогом по квантилю:

- строится kNN-модель на норме;
- вычисляется метрика аномальности для всех точек нормы;
- устанавливается порог по верхнему квантилю (99-й, 99.5-й и т. п.);
- на отложенных «нормальных» данных проверяется устойчивость порога при разных значениях k;
- найденный порог применяется к новым данным и анализируются ложные срабатывания.

Полезные эмпирические правила:

- Часто k берут порядка квадратного корня из размера выборки $k \approx \sqrt{n}$. Это сглаживает шум, но не слишком размывает локальную структуру.
- Доля аномалий: если ожидаемая доля аномалий небольшая (порядка 1%), то k должно быть достаточно большим, чтобы детектор не реагировал на случайные одиночные выбросы, но достаточно малым, чтобы не детектор не пропускал редкие аномальные режимы.
- Размер кластеров: если нормальный трафик образует плотные кластеры, k должен быть больше размера «шумовых» групп точек, но меньше размера типичного кластера.

Шаг 4. Выбор метрики расстояния.

Метрика, так же как и количество ближайших соседей, является настраиваемым параметром. Для его выбора рекомендуется использовать экспериментальные подходы на пример описанного выше для выбора значения k.

Наиболее часто применяются следующие метрики:

- манхэттенское расстояние;
- расстояние Минковского;

- расстояние Махаланобиса;
- косинусное расстояние.

Евклидово расстояние:

$$d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{m} (x_j - y_j)^2},$$

где:

- $\mathbf{x}$ — первый вектор признаков;
- $\mathbf{y}$ — второй вектор признаков;
- x_j — значение j-го признака объекта $\mathbf{x}$;
- y_j — значение j-го признака объекта $\mathbf{y}$;
- m — количество признаков.

Манхэттенское расстояние:

$$d_1(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^{d} |x_j - y_j|.$$

Расстояние Минковского.

Евклидово и манхэттенское расстояния являются частными случаями расстояния Минковского:

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^{d} |x_j - y_j|^p \right)^{1/p},$$

где:

- p — степень метрики;
- $p = 1$ соответствует манхэттенскому расстоянию;
- $p = 2$ соответствует евклидову расстоянию.

Чем сильнее различаются признаки двух сетевых событий, тем больше расстояние между ними.

Шаг 5. Поиск ближайших соседей.

Для анализируемого объекта $\mathbf{x}$ вычисляются расстояния до всех объектов обучающей выборки:

$$d(\mathbf{x}, \mathbf{x}_1), d(\mathbf{x}, \mathbf{x}_2), \ldots, d(\mathbf{x}, \mathbf{x}_i), \ldots, d(\mathbf{x}, \mathbf{x}_n),$$

где:

- $\mathbf{x}$ — анализируемый объект;
- $\mathbf{x}_i$ — i-й объект обучающей выборки;
- $d(\mathbf{x}, \mathbf{x}_i)$ — расстояние от анализируемого объекта до объекта $\mathbf{x}_i$;

- n — размер обучающей выборки.

Расстояния упорядочиваются по возрастанию:

$$d_{(1)}(\mathbf{x}) \leq d_{(2)}(\mathbf{x}) \leq \ldots \leq d_{(n)}(\mathbf{x}).$$

Множество k ближайших соседей для объекта $\mathbf{x}$ обозначается:

$$N_k(\mathbf{x}) = \{\mathbf{x}_{(1)}, \mathbf{x}_{(2)}, \ldots, \mathbf{x}_{(q)} \ldots, \mathbf{x}_{(k)}\},$$

где:

- $N_k(\mathbf{x})$ — множество k ближайших соседей объекта $\mathbf{x}$;
- $\mathbf{x}_{(q)}$ — q-й ближайший сосед;
- k — количество учитываемых соседей.

Шаг 6. Исключение самого объекта.

Этот шаг особенно важен в режиме обучения без учителя.

Если поиск соседей выполняется для объекта из той же выборки, по которой построен индекс, то ближайшим объектом будет он сам:

$$d(\mathbf{x}_i, \mathbf{x}_i) = 0.$$

Поэтому его необходимо исключить из списка соседей, иначе его нулевое расстояние искусственно уменьшит оценку аномальности.

В программной реализации обычно запрашивают $k + 1$ соседа и удаляют первого соседа с нулевым расстоянием.

При анализе действительно нового события сам объект отсутствует в эталонной выборке, поэтому дополнительное исключение не требуется.

Шаг 7. Вычисление оценки аномальности.

Существует несколько вариантов оценки.

Вариант 1. Расстояние до k-го соседа:

$$S_k(\mathbf{x}) = d_{(k)}(\mathbf{x}),$$

где:

- $d_{(k)}(\mathbf{x})$ — расстояние от $\mathbf{x}$ до k-го соседа;
- k — количество соседей;
- $\mathbf{x}$ — анализируемое событие.

Чем больше расстояние до k-го соседа, тем более изолирован объект.

Преимущество этой оценки — простота. Недостаток — результат зависит только от одного, k-го соседа, хотя поиск выполнялся для всех k соседей.

Вариант 2. Среднее расстояние до k соседей:

$$S_{\text{mean}}(\mathbf{x}) = \frac{1}{k} \sum_{q=1}^{k} d_{(q)}(\mathbf{x}).$$

Это более устойчивый вариант: среднее расстояние учитывает информацию обо всех k соседях и меньше зависит от одного случайного объекта. Если все ближайшие соседи находятся далеко, средняя оценка будет большой.

Вариант 3. Взвешенное среднее расстояние.

При использовании среднего расстояния все k ближайших соседей одинаково влияют на оценку аномальности. Однако ближайшие соседи обычно лучше характеризуют локальное окружение объекта, чем соседи, расположенные ближе к границе выбранной окрестности. Поэтому расстояниям можно назначить разные веса.

Ближайшим соседям можно назначить больший вес:

$$S_{\text{weighted}}(\mathbf{x}) = \frac{\sum_{q=1}^{k} w_q \cdot d_{(q)}(\mathbf{x})}{\sum_{q=1}^{k} w_q},$$

где:

- $S_{\text{weighted}}(\mathbf{x})$ — взвешенная оценка аномальности;
- w_q — вес расстояния до q-го ближайшего соседа;
- $d_{(q)}(\mathbf{x})$ — расстояние до q-го соседа;
- q — порядковый номер соседа;
- k — количество учитываемых соседей.

Чем больше вес w_q, тем сильнее соответствующий сосед влияет на итоговую оценку.

Самым простым способом определения веса — назначить веса по порядковому номеру соседа:

$$w_q = \frac{1}{q}.$$

Ближайшие соседи получают больший вес.

В базовом алгоритме обычно достаточно расстояния до k-го соседа или среднего расстояния.

Шаг 8. Выбор порога принятия решения об анамальности.

Процедура выбора порога отличается для различных режимов обучения.

Выбор порога в режиме обучения без учителя.

1. Получение оценок всей выборки.

Для каждого объекта вычисляется оценка аномальности:

$$S_i = S(\mathbf{x}_i), \qquad i = 1, \ldots, n,$$

где:

- S_i — оценка аномальности i-го события;
- $S(\mathbf{x}_i)$ — выбранная функция оценки, например среднее расстояние;
- $\mathbf{x}_i$ — i-й объект;
- n — количество объектов.

Поскольку метки отсутствуют, порог часто выбирается на основе предполагаемой доли аномалий α:

$$T_{\text{unsup}} = Q_{1-\alpha}\left(S_1, S_2, \ldots, S_n\right),$$

где:

- T_{unsup} — порог в режиме без учителя;
- α — предполагаемая доля аномалий;
- $1 - \alpha$ — уровень используемого квантиля;
- $Q_{1-\alpha}$ — квантиль уровня $1 - \alpha$;
- S_i — оценка аномальности i-го объекта.

Например, если $\alpha = 0{,}05$, порог равен 95-му процентилю оценок. Приблизительно 5% объектов будут объявлены аномальными.

2. Формирование метки.

$$\hat{y}_i = \begin{cases} 1, & S_i > T_{\text{unsup}}, \\ 0, & S_i \le T_{\text{unsup}}. \end{cases}$$

где:

- $\hat{y}_i$ — предсказанная метка i-го объекта;
- $\hat{y}_i = 1$ — обнаружена аномалия;
- $\hat{y}_i = 0$ — объект признан нормальным.

Такой порог не означает, что 5% событий действительно являются атаками. Он только заставляет метод пометить примерно 5% объектов выборки как аномальные. Если фактическая доля атак меньше, возрастает количество ложных тревог. Если больше — часть атак будет пропущена.

Выбор порога при обучении только на нормальных данных.

1. Разделение нормальных данных.

Эталонные нормальные данные следует разделить на обучающую и колибровочную выборки:

$$X_{\text{normal}} = X_{\text{fit}} \cup X_{\text{calibration}}, \qquad X_{\text{fit}} \cap X_{\text{calibration}} = \varnothing,$$

где:

- X_{normal} — все доступные нормальные данные;
- X_{fit} — нормальные данные, сохраняемые как эталонные соседи;
- $X_{\text{calibration}}$ — отложенные нормальные данные для выбора порога.

2. Построение эталонного набора.

Модель ближайших соседей обучается по нормальным обучающим событиям X_{fit}.

3. Оценивание калибровочных объектов.

Для каждого калибровочного объекта $\mathbf{x}_i^c$ вычисляется оценка аномальности (например, среднее расстояние до юлижайших соседей):

$$S_i^c = \frac{1}{k} \sum_{q=1}^{k} d\left(\mathbf{x}_i^c, \mathbf{x}_q^N\right),$$

где:

- c — обозначение калибровочной выборки;
- N — обозначение нормальных эталонных данных;
- S_i^c — оценка аномальности i-го калибровочного события;
- $\mathbf{x}_i^c$ — i-й калибровочный нормальный объект;
- $\mathbf{x}_q^N$ — q-й ближайший объект из нормальной обучающей выборки.

Калибровочный объект не входит в `X_fit`, поэтому исключать его из списка соседей не требуется.

3. Выбор порога.

$$T_{\text{normal}} = Q_{1-\alpha}\left(S_1^c, S_2^c, \ldots, S_{n_c}^c\right),$$

где:

- T_{normal} — порог, полученный по нормальным данным;
- α — допустимая доля ложных тревог;
- $Q_{1-\alpha}$ — квантиль уровня $1-\alpha$;
- S_i^c — оценка i-го калибровочного нормального события;
- n_c — количество калибровочных объектов.

Например, при $\alpha = 0{,}01$:

$$T_{\text{normal}} = Q_{0{,}99}(S^c).$$

Это означает, что примерно 99% калибровочных нормальных событий имеют оценку не выше порога.

Параметр α задаёт целевую частоту ложных тревог, но не гарантирует её точное соблюдение на будущих данных.

Шаг 9. Анализ нового события.

Пусть поступило новое событие: $\mathbf{x}_{new}$.

Сначала оно преобразуется параметрами масштабирования, полученными по обучающим данным:

$$z_{new,j} = \frac{x_{new,j} - \mu_j^N}{\sigma_j^N}.$$

Затем вычисляется среднее расстояние до k эталонных нормальных объектов:

$$S(\mathbf{x}_{new}) = \frac{1}{k} \sum_{q=1}^{k} d\left(\mathbf{x}_{new}, \mathbf{x}_q^N\right).$$

Итоговая метка определяется следующим образом:

$$\hat{y}_{new} = \begin{cases} 1, & S(\mathbf{x}_{new}) > T_{\text{normal}}, \\ 0, & S(\mathbf{x}_{new}) \le T_{\text{normal}}. \end{cases}$$

Основные различия режимов обучения.

Этап	Без учителя	Только на нормальных данных
Эталонные соседи	Все неразмеченные события	Только нормальные обучающие события
Наличие атак среди соседей	Возможно	Не предполагается
Масштабирование	По смешанной выборке	Только по нормальной обучающей выборке
Оценивание обучающего объекта	Сам объект исключается	Новое событие сравнивается с `normal_fit`
Выбор порога	По ожидаемой доле аномалий	По отложенным нормальным данным
Интерпретация порога	Бюджет тревог	Приближённый контроль ложных тревог
Риск маскирования атак	Высокий для групповых атак	Существенно ниже
Основное требование	Репрезентативная смешанная выборка	Чистая и полная выборка нормы

Реализация kNN в sklearn

В `sklearn` нет отдельного класса, для реализации алгоритма поиска аномалий методом kNN. Для поиска ближайших соседей используется класс `NearestNeighbors` модуля `sklearn.neighbors`.

Основные параметры:

- `n_neighbors` (по умолчанию 5): число ближайших соседей;
- `algorithm` (по умолчанию `auto`): алгоритм поиска соседей (`ball_tree`, `kd_tree`, `brute`);
- `metric` (по умолчанию `minkowski`): метрика расстояния;
- `p` (по умолчани 2): степень для метрики Минковского. Работает при выборе `metric=minkowski`: p=1 → манхэттен, p=2 → евклид.

Особенности класса `NearestNeighbors`:

- сохраняет эталонные объекты;
- находит ближайших соседей;
- возвращает расстояния и индексы соседей;
- не вычисляет оценку аномальности;
- не выбирает порог;
- не возвращает метки «норма/аномалия».

Поэтому оценку аномальности и порог необходимо реализовать самостоятельно.

Рассмотрим технологию детекции аномалий с использованием kNN и класса `NearestNeighbors`.

In [8]:
```
df.head()
```

Out[8]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	sce
0	S0000	49.93	10.93	28.74	web	n
1	S0001	41.26	13.53	30.24	web	n
2	S0002	53.06	13.54	25.97	web	n
3	S0003	64.45	14.59	24.73	web	n
4	S0004	58.31	13.76	19.01	web	n

Структура набора данных:

```
df
├── reference_normal — 600 подтверждённых нормальных сессий
└── monitoring       — 254 новые сессии
    ├── 240 нормальных сессий
    └── 14 атак
        ├── 4 port_scan
        ├── 4 beaconing
        ├── 3 exfiltration
        └── 3 lateral_movement
```

Используем признаки.

In [9]:
```
FEATURES = ["connections_per_min",
            "unique_destinations",
            "outbound_mb"]
```

Реализация при обучении без учителя

В режиме обучения без учителя kNN строится по всей неразмеченной мониторинговой выборке `monitoring`. В этой выборке находятся как нормальные события, так и атаки.

In [10]:
```
monitoring_df.head()
```

Out[10]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	s
600	S0600	56.50	15.11	22.97	web	
601	S0601	57.21	19.07	26.54	web	
602	S0602	51.52	13.02	22.68	web	
603	S0603	40.72	14.97	30.16	web	
604	S0604	53.06	13.11	23.70	web	

Модель не знает, какие объекты являются атаками. Нормальные и атакующие события могут становиться соседями.

В рассматриваемом примере модель одновременно:

- запоминает мониторинговые события;
- оценивает аномальность этих же событий.

Поэтому при поиске соседей необходимо исключить сам анализируемый объект.

Визуализируем распределение мониторинговых данных.

Три признака невозможно одновременно представить на обычном двумерном графике, поэтому построим две проекции мониторинговой выборки:

- количество соединений в минуту и количество уникальных адресов назначения;
- количество соединений в минуту и объём исходящего трафика.

Каждая проекция показывает только два из трёх признаков, поэтому не отражает полностью взаимное расположение объектов в трёхмерном пространстве.

In [11]:
```
scenario_colors = {"normal": "steelblue",
                   "port_scan": "red",
                   "exfiltration": "darkorange",
                   "beaconing": "purple",
                   "lateral_movement": "green"}
```

```
fig, axes = plt.subplots(1, 2, figsize = (7, 5))

for scenario, group in monitoring_df.groupby("scenario"):
    axes[0].scatter(group["connections_per_min"],
                    group["unique_destinations"],
                    color = scenario_colors[scenario],
                    label = scenario, alpha = 0.75, s = 45)

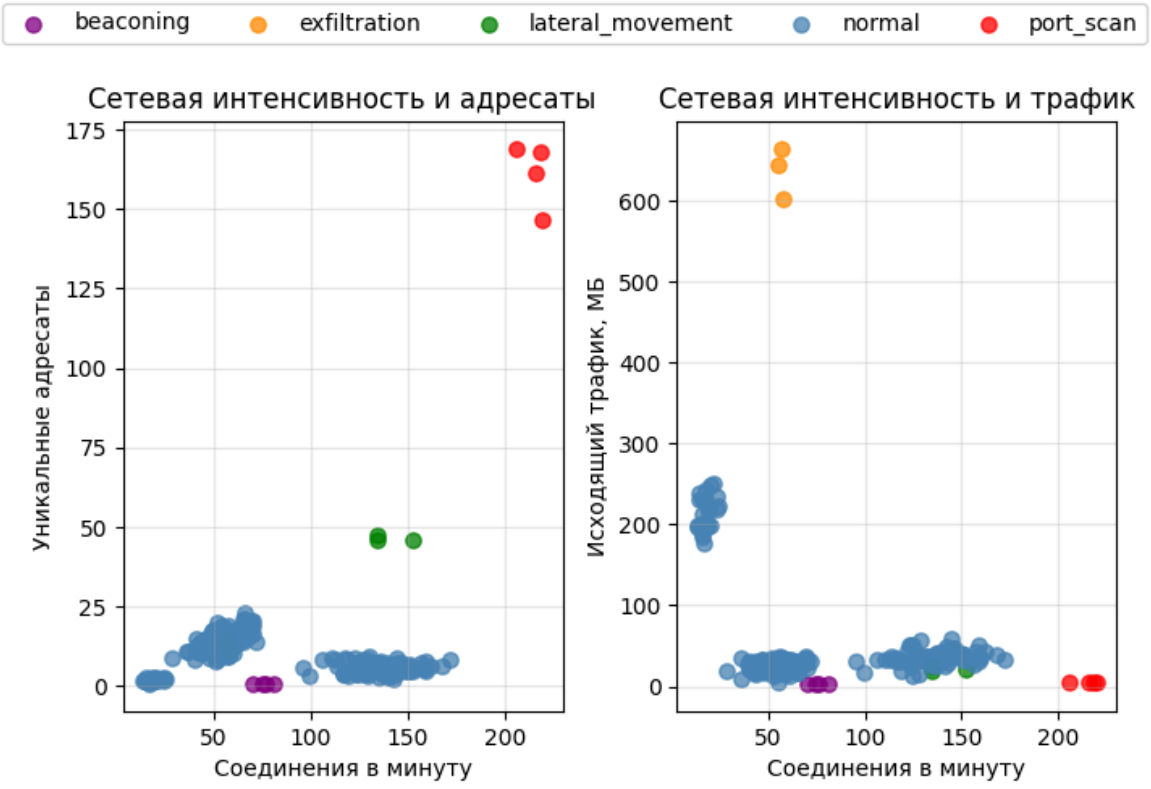
    axes[1].scatter(group["connections_per_min"],
                    group["outbound_mb"],
                    color = scenario_colors[scenario],
                    label = scenario, alpha = 0.75, s = 45)

axes[0].set_xlabel("Соединения в минуту")
axes[0].set_ylabel("Уникальные адресаты")
axes[0].set_title("Сетевая интенсивность и адресаты")
axes[0].grid(alpha = 0.3)

axes[1].set_xlabel("Соединения в минуту")
axes[1].set_ylabel("Исходящий трафик, МБ")
axes[1].set_title("Сетевая интенсивность и трафик")
axes[1].grid(alpha = 0.3)

handles, labels = axes[0].get_legend_handles_labels()

fig.legend(handles, labels, loc = "upper center", ncols = 5)
plt.tight_layout(rect = [0, 0, 1, 0.92])
```



На графиках видно, что разные типы активности образуют разные «облака» в пространстве признаков.

Левый график:

- `normal` (синие точки) — два характерных режима: основной кластер (40–60 соединений, 5–25 адресатов) — обычное взаимодействие с разными узлами,

полоса точек у нуля (100–180 соединений, ~0 адресатов) — высокочастотное общение с одним и тем же адресатом (например, база данных, к которой приложение обращается постоянно);

- `port_scan` (красные точки) — аномалии: ~200+ соединений в минуту при 145–170 уникальных адресатах. Сканер опрашивает все порты/хосты подряд — отсюда экстремально много соединений и много разных адресов;
- `lateral_movement` (зелёные точки) — кластер на умеренных ~130–160 соединениях, но 45–50 уникальных адресатов. Такая совокупность значений характерина для бокового перемещения: вредоносный процесс (или скомпрометированный аккаунт) за короткое время опрашивает много разных внутренних хостов (отсюда большое число уникальных адресатов), делает это активно, но не с такой большой частотой, как `port_scan` — потому что распространение идёт по логике сети, а не по перебору всех портов подряд. Это поведение нехарактерно для легитимной машины: обычная рабочая станция общается с ограниченным набором серверов, а не с полусотней узлов за минуту;
- `beaconing` (фиолетовый) — на этом графике почти незаметен: малое число соединений и почти ноль уникальных адресатов — обращение к одному внешнему C2 серверу злоумышленника.

Правый график:

- `normal` (синие точки) — вся норма "прижата" к нулю по оси трафика: даже при 180 соединениях в минуту исходящий объём мал (запросы и ответы вписываются в обычный профиль); `exfiltration` (оранжевые точки) — наиболее наглядный выброс: ~45–50 соединений в минуту, но 600–670 МБ исходящего трафика. Классическая сигнатура кражи данных;
- `beaconing` (фиолетовые точки) — также почти невидим: мало соединений, нулевой трафик (C2-пинги маленькие). Виден лишь как тонкая полоска около нуля — и снова «растворяется» в норме;
- `port_scan` (красные точки) — сканер только опрашивает порты, данных не передаёт, зато по числу соединений — экстремум.

Создадим вспомогательную функцию для расчёта оценки аномальности события.

Реализуем два варианта оценки:

- `mean` — среднее расстояние до k соседей;
- `kth` — расстояние до k-го соседа.

In [12]:
```
def calculate_knn_anomaly_score(distances,
                                score_type = "mean"):
    """
    distances : ndarray, shape (n_objects, k)
        содержит расстояния от одного объекта
        до его k ближайших соседей.
    """
    if score_type == "mean":
        # Среднее расстояние до всех k соседей.
```

```
        scores = distances.mean(axis = 1)

    elif score_type == "kth":
        # kneighbors() возвращает расстояния,
        # отсортированные по возрастанию.
        # Поэтому последний столбец соответствует
        # расстоянию до k-го соседа.
        scores = distances[:, -1]

    else:
        raise ValueError("score_type должен быть 'mean' или 'kth'")
    return scores
```

Масштабирование признаков.

In [13]:
```
# Матрица мониторинговых признаков.
X_unsupervised = monitoring_df[FEATURES].to_numpy()

# Истинные метки нужны только для последующей оценки.
# Они не передаются в StandardScaler и NearestNeighbors.
y_monitoring = monitoring_df["is_attack"].to_numpy()

# В режиме без учителя отдельной нормальной выборки нет.
# Поэтому параметры масштабирования рассчитываются
# по всей неразмеченной мониторинговой выборке.
scaler_unsupervised = StandardScaler()

X_unsupervised_scaled = (scaler_unsupervised.fit_transform(X_unsupervised))
X_unsupervised_scaled[:5]
```

Out[13]:
```
array([[-0.47948747,  0.09505378, -0.35035616],
       [-0.46380287,  0.29426388, -0.30828921],
       [-0.58950058, -0.01008488, -0.35377337],
       [-0.82808324,  0.088011  , -0.26563308],
       [-0.55548046, -0.00555738, -0.34175424]])
```

Создание модели ближайших соседей.

Метод `fit(X, y=None)` обучает модель:

- сохраняет объекты, среди которых в дальнейшем будут искаться соседи;
- в зависимости от `algorithm='auto'/'ball_tree'/'kd_tree'/'brute'` строит структуру (KDTree, BallTree или просто хранит матрицу), чтобы быстро находить соседей.

In [14]:
```
from sklearn.neighbors import NearestNeighbors

# Выберем количество ближайших соседей равным 10
K_NEIGHBORS = 10

knn_unsupervised = NearestNeighbors(n_neighbors = K_NEIGHBORS,
                                    metric = "euclidean",
                                    # Scikit-learn самостоятельно выбирает спосо
                                    # brute force, KDTree или BallTree.
                                    algorithm = "auto")

# Метки классов в fit() не передаются.
knn_unsupervised.fit(X_unsupervised_scaled)
```

Out[14]:
```
NearestNeighbors
NearestNeighbors(metric='euclidean', n_neighbors=10)
```

Поиск соседей с исключением самого объекта.

Для поиска ближайших соседей используется метод

`kneighbors(X=None, n_neighbors=None, return_distance=True)` :

- `X` : точки, для которых находятся ближайшие соседи. Если `None` , поиск ведётся по тем же данным, на которых делали `fit()` (то есть по обучающей выборке);
- `n_neighbors` : число ближайших соседей для возврата. По умолчанию `None` - берётся значение, заданное при создании модели `n_neighbors=k` ;
- `return_distance` :
  - если `True` , возвращает массив расстояний;
  - если `False` возвращает только индексы соседей. Для детекции аномалий всегда нужен `return_distance=True` , потому что матрика аномальности строится на расстояниях.

Метод возвращает кортеж: `(distances, indices)` :

- `distances` (тип `ndarray` формы `(n_samples_query, n_neighbors)`): содержит расстояния от каждой точки запроса до её ближайших соседей (в порядке возрастания расстояния);
- `indices` (тип `ndarray` формы `(n_samples_query, n_neighbors)`): содержит индексы соседей в обучающей выборке. Полезен для интерпретации: можно посмотреть, какие именно сессии/потоки считаются «похожими» на подозрительную точку, и понять, к какому режиму трафика она относится.

In [15]:
```
# X не передаётся в kneighbors().
# Будем определять ближайших соседей для каждого
# объекта мониторинговой выборки
distances_unsupervised, indices_unsupervised = (knn_unsupervised.kneighbors())

print(f'Размер мониторинговой выборки: {len(X_unsupervised)}')
print(f'Размер массива расстояний: {distances_unsupervised.shape}')
print(f'Размер массива индексов: {indices_unsupervised.shape}')
```

```
Размер мониторинговой выборки: 254
Размер массива расстояний: (254, 10)
Размер массива индексов: (254, 10)
```

Каждая строка содержит расстояния до десяти ближайших соседей для одного события.

In [16]:
```
distances_unsupervised
```

Out[16]:
```
array([[0.01282144, 0.03097236, 0.03692204, ..., 0.0667725 , 0.06910939,
        0.08419971],
       [0.06980414, 0.08592291, 0.0877406 , ..., 0.12446537, 0.13208763,
        0.13318512],
       [0.03050526, 0.0363638 , 0.04059564, ..., 0.06719483, 0.06821714,
        0.07090825],
       ...,
       [0.09348098, 0.39530064, 1.86489424, ..., 1.93402112, 1.93799136,
        1.93850031],
       [0.09348098, 0.40607954, 1.9262416 , ..., 1.98875252, 1.98943835,
        1.99171478],
       [0.39530064, 0.40607954, 1.90597528, ..., 1.9718533 , 1.97418679,
        1.98777151]], shape=(254, 10))
```

Каждая строка содержит индексы десяти ближайших соседей для одного события.

In [17]:
```
indices_unsupervised
```

Out[17]:
```
array([[ 89,  78, 132, ...,  64,  54, 143],
       [ 92, 131,  91, ...,  86,  53, 123],
       [ 55,   4,  82, ...,  95, 140, 111],
       ...,
       [252, 253, 202, ..., 206, 196,  28],
       [251, 253, 202, ..., 206,  15, 148],
       [251, 252, 201, ..., 184, 181, 198]], shape=(254, 10))
```

Расчёт оценок аномальности.

In [18]:
```
knn_unsupervised_scores = (
    calculate_knn_anomaly_score(distances = distances_unsupervised,
                                score_type = 'mean'))
knn_unsupervised_scores[:5]
```

Out[18]:
```
array([0.05114465, 0.10425312, 0.05454707, 0.14131203, 0.05054752])
```

Выбор порога.

В режиме без учителя нет отдельной нормальной калибровочной выборки. Поэтому зададим предполагаемую долю аномалий.

In [19]:
```
# Экспертное предположение:
# около 5,5% событий могут быть аномальными.
#
# В реальной системе это значение не должно вычисляться
# с использованием истинного столбца is_attack.
EXPECTED_ANOMALY_FRACTION = 0.055

knn_unsupervised_threshold = np.quantile(knn_unsupervised_scores,
                                         1 - EXPECTED_ANOMALY_FRACTION)
```

```
print(f'Порог kNN в режиме без учителя: {(knn_unsupervised_threshold).round(3)}'
```

```
Порог kNN в режиме без учителя: 0.421
```

Получение предсказаний.

In [20]:
```
# Формируем метки: 1 - аномалия, 0 - норма
knn_unsupervised_prediction = (
    knn_unsupervised_scores > knn_unsupervised_threshold).astype(int)

# Сохраняем результаты в таблицу.
monitoring_df["knn_unsupervised_score"] = (knn_unsupervised_scores)
monitoring_df["knn_unsupervised_anomaly"] = (knn_unsupervised_prediction)

monitoring_df[[*FEATURES + ["knn_unsupervised_score", "knn_unsupervised_anomaly"
```

Out[20]:

	connections_per_min	unique_destinations	outbound_mb	knn_unsupervised_score
600	56.50	15.11	22.97	0.051145
601	57.21	19.07	26.54	0.104253
602	51.52	13.02	22.68	0.054547
603	40.72	14.97	30.16	0.141312
604	53.06	13.11	23.70	0.050548

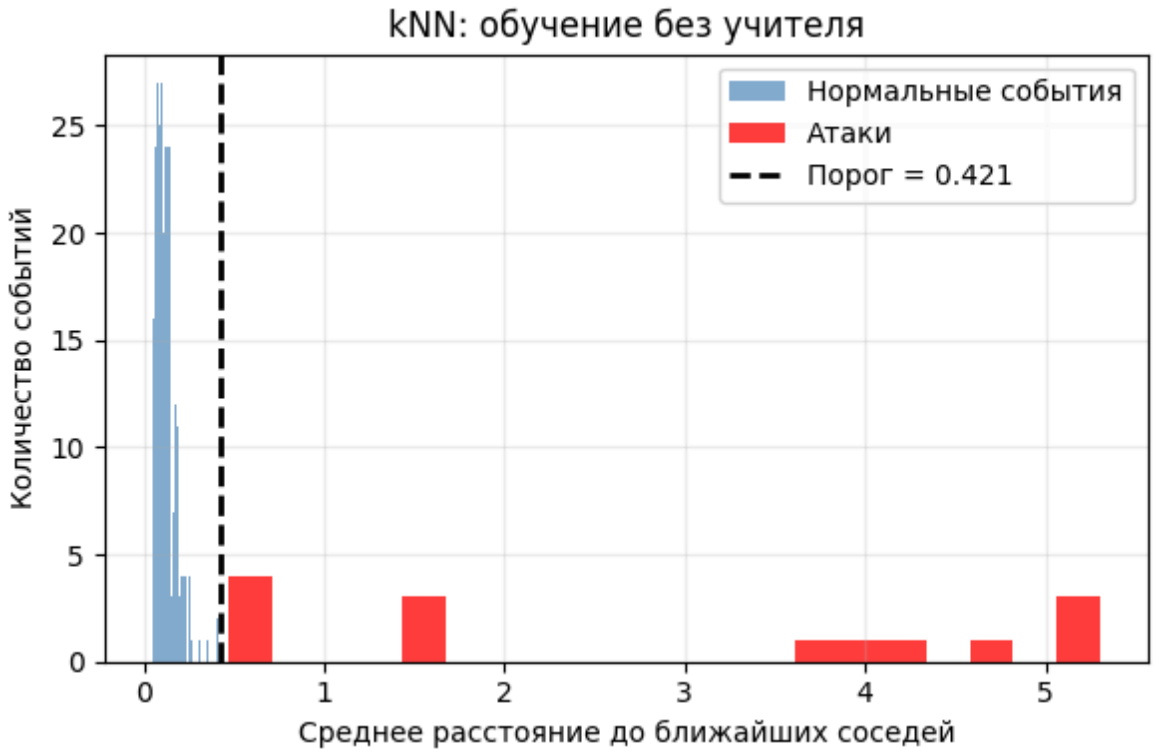
Оценивание качества детекции аномалий.

In [21]:
```
print_detection_result(method_name = "kNN: обучение без учителя",
                       y_true = monitoring_df["is_attack"],
                       y_pred = monitoring_df["knn_unsupervised_anomaly"])
```

```
kNN: обучение без учителя
TP — обнаруженные атаки: 14
FP — ложные тревоги: 0
FN — пропущенные атаки: 0
TN — корректно распознанная норма: 240
```

Визуализация режима обучения без учителя.

In [34]:
```
plot_knn_scores(data = monitoring_df,
                score_column = "knn_unsupervised_score",
                threshold = knn_unsupervised_threshold,
                title = "kNN: обучение без учителя")
```



Подавляющее большинство нормальных событий имеет очень маленькие значения метрики аномальности — от 0 до 0.2. Каждая нормальная сессия — это одна из точек внутри показанных выше облаков. Внутри таких облаков точки лежат плотно, поэтому для любой нормальной сессии её k ближайших соседей — такие же нормальные сессии из того же облака — находятся буквально рядом.

Небольшая часть нормальных событий — края облаков, где точки реже. Их ближайшие соседи чуть дальше, отсюда и метрика до 0.4. Но все они, тем не менее, остаются левее порога 0.421: модель ни одну из них не отметила как аномалию.

Красные столбцы разбросаны тремя отдельными группами: около 0.5–0.7, около 1.6–1.8 и далеко справа (3.5–5). Каждая группа — это отдельный тип атаки со своим «удалением» от нормы:

- ближайшая группа (~0.5–0.7) — атаки, маскирующиеся под норму (`beaconing` : малое число соединений и трафика — он лишь чуть-чуть отстоит от синего облака);
- средняя (~1.6) — умеренно нетипичная активность (`lateral_movement`);
- крайние (3.5–5) — самые экстремальные по признакам (`port_scan` , `exfiltration`).

Красная группа у 0.5–0.7 — зона риска. Она вплотную к порогу: небольшое изменение данных, дрейф трафика или переобучение — и объекты этой группы могут попасть по разные стороны порога.

Реализация при обучении только на нормальных данных

В этом режиме используется только нормальная выборка `reference_normal`.

Визуализируем распределение нормальных данных.

In [23]:
```
# Цвета нормальных режимов трафика.
traffic_colors = {"web": "#4C78A8", "api": "#F58518", "backup": "#54A24B"}

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for traffic_type, color in traffic_colors.items():

    # Выбираем нормальные события конкретного режима.
    part = reference_df[reference_df["traffic_type"].eq(traffic_type)]

    # Первая проекция:
    # частота соединений и число адресатов.
    axes[0].scatter(part["connections_per_min"],
                    part["unique_destinations"],
                    s = 25, alpha = 0.5, color = color, label = traffic_type)

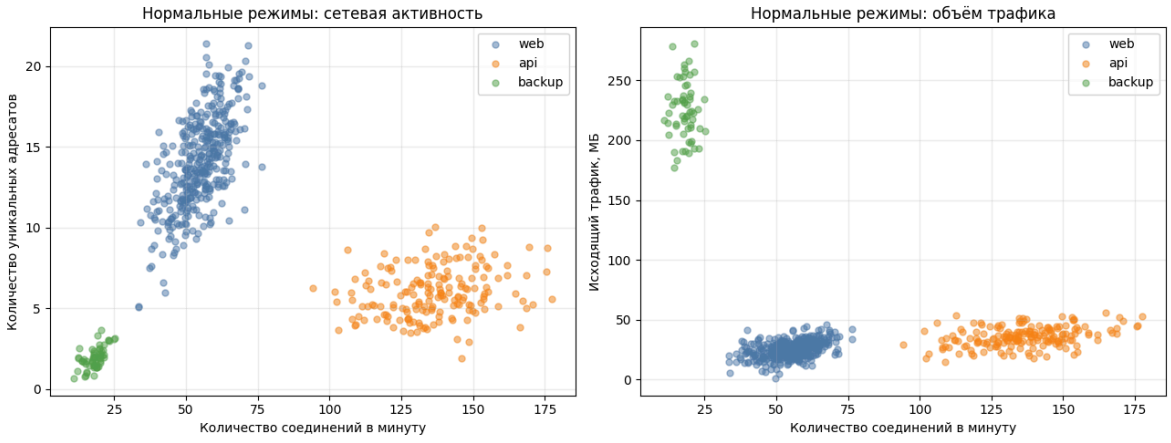
    # Вторая проекция:
    # частота соединений и исходящий трафик.
    axes[1].scatter(part["connections_per_min"],
                    part["outbound_mb"],
                    s = 25, alpha = 0.5, color = color, label = traffic_type)

axes[0].set(xlabel = "Количество соединений в минуту",
            ylabel = "Количество уникальных адресатов",
            title = "Нормальные режимы: сетевая активность")

axes[1].set(xlabel = "Количество соединений в минуту",
            ylabel = "Исходящий трафик, МБ",
            title = "Нормальные режимы: объём трафика")

for ax in axes:
    ax.grid(alpha = 0.25)
    ax.legend()

plt.tight_layout()
```



Из рисунков видно, что режимы хорошо разделимы. Это обеспечивает низкий уровень ложных срабатываний. Чтобы легитимная сессия получила высокую оценку аномальности, она должна оказаться далеко от всех точек нормы — а не просто «между режимами».

В этом режиме нормальную выборку нужно разделить на две части:

- `normal_fit` — построение модели;
- `normal_calibration` — выбор порога.

Разделение необходимо, потому что расстояния объектов из `normal_fit` до других объектов `normal_fit` обычно меньше, чем расстояния действительно новых нормальных событий. Порог, рассчитанный на тех же данных, может оказаться слишком низким.

`reference_df` — датафрейм, который содержит только нормальные наблюдения. Будет использоваться для обучения модели.

`monitoring_df` — датафрейм, который содержит как нормальные наблюдения, так и атаки. Будет использоваться для проверки новых событий.

Разделение нормальных данных.

In [24]:
```
normal_fit_df, normal_calibration_df = train_test_split(
    reference_df,

    # 25% нормальных данных оставляем
    # для выбора порога.
    test_size = 0.25,
    random_state = 42,

    # Сохраняем пропорции нормальных режимов:
    # web, api и backup.
    stratify = reference_df["traffic_type"])

# Сбрасываем индексы, чтобы позиции соседей,
# возвращаемые kneighbors(), совпадали с iloc.
normal_fit_df = normal_fit_df.reset_index(drop = True)
normal_calibration_df = (normal_calibration_df.reset_index(drop = True))

print("Норма для построения модели:", len(normal_fit_df))
print("Норма для калибровки:", len(normal_calibration_df))
```

```
Норма для построения модели: 450
Норма для калибровки: 150
```

Масштабирование признаков.

Масштабировать признаки нужно только по нормальной обучающей выборке `normal_fit`.

In [25]:
```
X_normal_fit = (normal_fit_df[FEATURES].to_numpy())
X_normal_calibration = (normal_calibration_df[FEATURES].to_numpy())
X_monitoring = (monitoring_df[FEATURES].to_numpy())

# scaler обучается только на нормальных данных normal_fit.
scaler_normal = StandardScaler()
X_normal_fit_scaled = scaler_normal.fit_transform(X_normal_fit)

# Калибровочные и мониторинговые данные
```

```
# преобразуются уже обученным scaler.
#
# Здесь нельзя применять fit_transform(), потому что
# параметры масштабирования не должны пересчитываться.
X_normal_calibration_scaled = scaler_normal.transform(X_normal_calibration)
X_monitoring_scaled = scaler_normal.transform(X_monitoring)
```

Построение kNN только по нормальным данным.

In [26]:
```
knn_normal = NearestNeighbors(n_neighbors = K_NEIGHBORS,
                              metric = "euclidean",
                              algorithm = "auto")

# В качестве эталонных соседей сохраняются
# исключительно нормальные обучающие события.
knn_normal.fit(X_normal_fit_scaled)
```

Out[26]:
```
NearestNeighbors(metric='euclidean', n_neighbors=10)
```

Поиск соседей для объектов колибровочной выборки.

Калибровочные объекты имитируют новые легитимные сетевые события.

In [27]:
```
# Для каждого калибровочного нормального события
# ищем k соседей среди normal_fit.
#
# Калибровочные события не входят в normal_fit,
# поэтому исключать сам объект не требуется.
calibration_distances, calibration_indices = (
    knn_normal.kneighbors(X_normal_calibration_scaled))

knn_calibration_scores = (calculate_knn_anomaly_score(
        distances = calibration_distances, score_type = 'mean'))

knn_calibration_scores[:5]
```

Out[27]:
```
array([0.13474085, 0.10672382, 0.54227217, 0.10337825, 0.24909248])
```

Выбор порога по нормальным данным.

Выберем 99-й процентиль.

Порог имеет следующую интерпретацию:

Около 99% отложенных нормальных событий имеют среднее расстояние до соседей не выше выбранного порога.

Следовательно, при неизменном распределении нормального трафика целевая частота ложных тревог составляет примерно 1%.

In [28]:
```
NORMAL_SCORE_QUANTILE = 0.99

knn_normal_threshold = np.quantile(knn_calibration_scores,
                                   NORMAL_SCORE_QUANTILE)
```

```
print(f'Порог kNN при обучении только на норме: {(knn_normal_threshold).round(3)
```

```
Порог kNN при обучении только на норме: 0.457
```

Анализ мониторинговых событий.

In [29]:
```
# Ищем ближайших соседей каждого мониторингового события
# исключительно среди нормальных объектов normal_fit.
monitoring_distances_normal, monitoring_indices_normal = (
    knn_normal.kneighbors(X_monitoring_scaled))

knn_normal_scores = (calculate_knn_anomaly_score(
        distances = monitoring_distances_normal, score_type = 'mean'))

knn_normal_prediction = (knn_normal_scores > knn_normal_threshold).astype(int)

# Записываем результаты в общую таблицу.
monitoring_df["knn_normal_score"] = (knn_normal_scores)
monitoring_df["knn_normal_anomaly"] = (knn_normal_prediction)
monitoring_df.head()
```

Out[29]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	s
600	S0600	56.50	15.11	22.97	web	
601	S0601	57.21	19.07	26.54	web	
602	S0602	51.52	13.02	22.68	web	
603	S0603	40.72	14.97	30.16	web	
604	S0604	53.06	13.11	23.70	web	

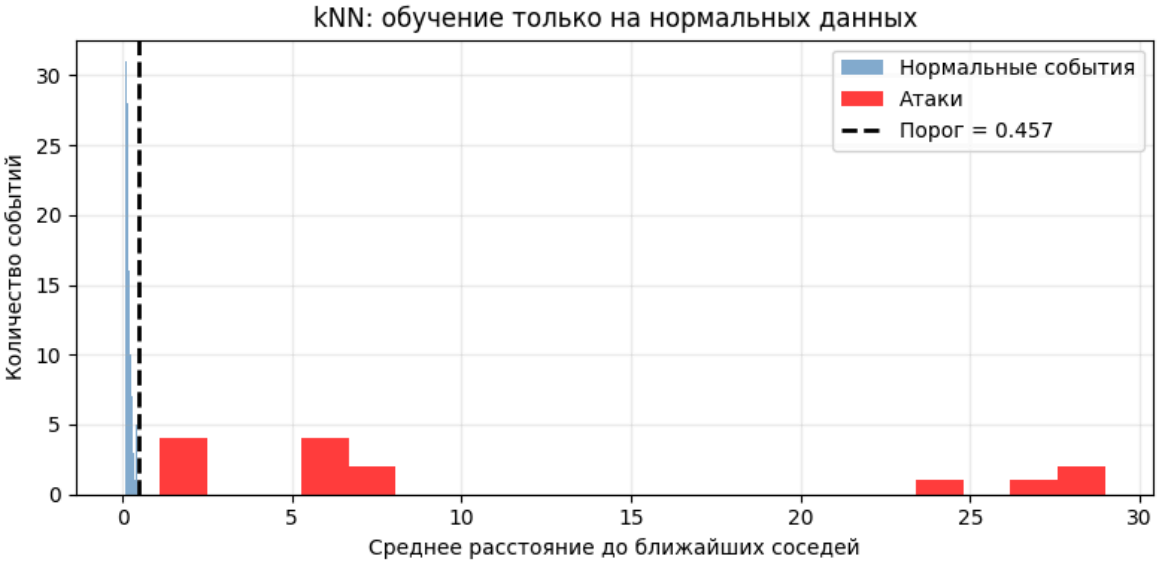
Оценивание качества детекции аномалий.

In [30]:
```
print_detection_result(method_name = ("kNN: обучение только на нормальных данных
                       y_true = monitoring_df["is_attack"],
                       y_pred = monitoring_df["knn_normal_anomaly"])
```

```
kNN: обучение только на нормальных данных
TP — обнаруженные атаки: 14
FP — ложные тревоги: 1
FN — пропущенные атаки: 0
TN — корректно распознанная норма: 239
```

Визуализация режима обучения только на норме.

In [32]:
```
plot_knn_scores(data = monitoring_df,
                score_column = "knn_normal_score",
                threshold = knn_normal_threshold,
                title = "kNN: обучение только на нормальных данных")
```



Выбор количества ближайших соседей.

В предыдущих примерах использовалось детерминированное количество ближайших соседей $k = 10$. На практике это гиперпараметр является настраиваемым и его значение нужно подбирать. Для выбора оптимального значения k будем сравнивать метрики качества детекции при различном числе ближайших соседей.

In [60]:
```
# Исследуемые значения параметра.
STUDY_K_VALUES = [3, 5, 10, 20, 40]

# Сохраняем правила определения порога.
STUDY_ANOMALY_FRACTION = 0.055
STUDY_NORMAL_QUANTILE = 0.99

# Метки нужны только для оценивания качества детекции.
# В матрицу признаков они не входят.
study_y = monitoring_df["is_attack"].to_numpy(dtype = bool)

print("Мониторинговых сессий:", len(monitoring_df))
print("Нормальных сессий:", int((~study_y).sum()))
print("Атакующих сессий:", int(study_y.sum()))
```

```
Мониторинговых сессий: 254
Нормальных сессий: 240
Атакующих сессий: 14
```

Режим обучения без учителя

Соседи ищутся среди всех мониторинговых сессий. Атакующие объекты тоже могут становиться соседями друг друга.

Стандартизацию рассчитываем один раз по смешанной выборке. Для всех вариантов k используется одно и то же стандартизированное пространство.

В качестве оценки аномальности будет использоваться среднее расстояние до k ближайших соседей.

In [61]:

```
study_unsup_scaler = StandardScaler()

# Стандартизация признаков мониторинговой выборки.
study_X_unsup = study_unsup_scaler.fit_transform(monitoring_df[FEATURES])

# Здесь будут строки итоговой таблицы.
study_unsup_rows = []

# Здесь сохраняем решения каждого варианта.
# Ключ словаря — k, значение — массив тревог.
study_unsup_predictions = {}

# Обучаем модель с различными значениями k.
for k in STUDY_K_VALUES:
    study_model = NearestNeighbors(n_neighbors = k,
                                   metric = "euclidean",
                                   algorithm = "auto")

    # Сохраняем всю смешанную выборку как набор соседей.
    study_model.fit(study_X_unsup)

    # X не передаём:
    # ищем соседей объектов обучающей выборки,
    # исключая сам анализируемый объект.
    study_distances, _ = study_model.kneighbors()

    # В каждой строке находятся k расстояний.
    # Получаем одну оценку на сессию.
    study_scores = study_distances.mean(axis = 1)

    # Для текущего k рассчитываем собственный порог,
    # сохраняя прежний уровень квантиля.
    study_threshold = np.quantile(study_scores, 1 - STUDY_ANOMALY_FRACTION)

    # Формируем метки аномальности событий (сигналы тревоги).
    study_alerts = study_scores > study_threshold
    study_unsup_predictions[k] = study_alerts

    # Истинные метки используем только после получения тревог.
    study_unsup_rows.append({"k": k,
                             "Порог": study_threshold,
                             "Всего атакующих сессий": int(study_y.sum()),
                             "Срабатываний": int(study_alerts.sum()),
                             "Правильно обнаружено": int((study_alerts & study_y
                             "Ложно обнаружено": int((study_alerts & ~study_y).s
                             "Пропущено": int((~study_alerts & study_y).sum())})

study_unsup_table = pd.DataFrame(study_unsup_rows)
study_unsup_table
```

Out[61]:

	k	Порог	Всего атакующих сессий	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
0	3	0.190106	14	14	10	4	4
1	5	0.293754	14	14	12	2	2
2	10	0.421441	14	14	14	0	0
3	20	0.524159	14	14	14	0	0
4	40	1.268219	14	14	10	4	4

Выбирается значение $k = 3$, которое обеспечивает лучшие метрики качества детекции.

Режим обучения только на нормальных данных

Теперь эталонными соседями становятся только события из `normal_fit_df`.

Для каждого k:

1. Оцениваем нормальные калибровочные сессии относительно эталона.
2. Рассчитываем порог по их 99-му процентилю.
3. Оцениваем мониторинговые сессии относительно того же эталона.
4. Подсчитываем обнаружения и ошибки.

In [62]:
```
# Стандартизацию обучаем только на нормальном эталоне.
study_normal_scaler = StandardScaler()
study_X_fit = study_normal_scaler.fit_transform(normal_fit_df[FEATURES])

# Калибровочные и мониторинговые данные преобразуем
# уже обученным scaler.
study_X_calibration = study_normal_scaler.transform(normal_calibration_df[FEATUR
study_X_monitoring = study_normal_scaler.transform(monitoring_df[FEATURES])

study_normal_rows = []
study_normal_predictions = {}

for k in STUDY_K_VALUES:
    study_model = NearestNeighbors(n_neighbors = k,
                                   metric = "euclidean",
                                   algorithm = "auto")

    # Соседями могут быть только нормальные эталонные сессии.
    study_model.fit(study_X_fit)

    # Оцениваем отдельные калибровочные объекты.
    # Они не входят в эталон, поэтому сам объект
    # дополнительно исключать не требуется.
    study_cal_distances, _ = study_model.kneighbors(study_X_calibration)
    study_cal_scores = study_cal_distances.mean(axis = 1)

    # Численный порог свой для каждого k.
```

```
    # Уровень квантиля остаётся равным 0.99.
    study_threshold = np.quantile(study_cal_scores, STUDY_NORMAL_QUANTILE)

    # Проверяем мониторинговые сессии.
    study_distances, _ = study_model.kneighbors(study_X_monitoring)
    study_scores = study_distances.mean(axis = 1)
    study_alerts = study_scores > study_threshold
    study_normal_predictions[k] = study_alerts

    study_normal_rows.append({"k": k,
                              "Порог": study_threshold,
                              "Всего атакующих сессий": int(study_y.sum()),
                              "Срабатываний": int(study_alerts.sum()),
                              "Правильно обнаружено": int((study_alerts & study_
                              "Ложно обнаружено": int((study_alerts & ~study_y).
                              "Пропущено": int((~study_alerts & study_y).sum())}

study_normal_table = pd.DataFrame(study_normal_rows)
study_normal_table
```

Out[62]:

	k	Порог	Всего атакующих сессий	Срабатываний	Правильно обнаружено	Ложно обнаружено	Пропущено
0	3	0.307586	14	18	14	4	0
1	5	0.347252	14	17	14	3	0
2	10	0.457301	14	15	14	1	0
3	20	0.561294	14	15	14	1	0
4	40	0.796128	14	15	14	1	0

Общее количество пропусков недостаточно для обоснования выбора значения k. Нужно проанализировать, какие сценарии атак были пропущены и какие нормальные режимы работы вызывают ложные тревоги.

Какие сценарии пропускаются.

In [65]:
```
study_scenario_rows = []

for regime, predictions in [("Без учителя", study_unsup_predictions),
                            ("Только норма", study_normal_predictions)]:
    for k in STUDY_K_VALUES:
        study_alerts = predictions[k]
        for scenario in ["port_scan",
                         "exfiltration",
                         "beaconing",
                         "lateral_movement"]:

            # Маска сессий одного атакующего сценария.
            scenario_mask = (monitoring_df["scenario"].eq(scenario).to_numpy())

            study_scenario_rows.append({"Режим": regime,
                                        "k": k,
                                        "Сценарий": scenario,
```

```
                                        "Всего": int(scenario_mask.sum()),
                                        "Обнаружено": int((study_alerts & scenar
                                        "Пропущено": int((~study_alerts & scenar

study_scenario_table = pd.DataFrame(study_scenario_rows)

# В ячейках таблицы — число обнаруженных сессий.
study_scenario_table.pivot(index = ["Режим", "k"],
                           columns = ["Сценарий", "Всего"],
                           values = "Обнаружено")
```

Out[65]:

	Сценарий	port_scan	exfiltration	beaconing	lateral_movement
	Всего	4	3	4	3
Режим	k				
Без учителя	3	4	3	0	3
	5	4	3	2	3
	10	4	3	4	3
	20	4	3	4	3
	40	4	3	0	3
Только норма	3	4	3	4	3
	5	4	3	4	3
	10	4	3	4	3
	20	4	3	4	3
	40	4	3	4	3

Изменение величины k влияет, прежде всего, на обнаружение атак `beaconing`.

При $k = 3$ группа из четырёх похожих сессий может поддерживать сама себя: у каждой сессии есть три близких соседа из той же группы. Поэтому небольшие расстояния внутри группы не позволяют ей выделиться среди верхних оценок.

При $k = 10$ и $k = 20$ оценивается более широкое окружение, и все четыре сессии вызывают тревоги.

При обучении только на норме все четыре сценария обнаруживаются при каждом исследованном k. Атакующие сессии не становятся эталонными соседями друг друга.

Какие нормальные режимы вызывают ложные тревоги.

In [66]:
```
study_false_rows = []

for regime, predictions in [("Без учителя", study_unsup_predictions),
                            ("Только норма", study_normal_predictions)]:
    for k in STUDY_K_VALUES:
        # Тревога есть, но сессия по разметке нормальная.
        false_mask = predictions[k] & ~study_y
```

```
        traffic_counts = (monitoring_df.loc[false_mask, "traffic_type"].value_co

        study_false_rows.append({"Режим": regime,
                                 "k": k,
                                 "web": int(traffic_counts.get("web", 0)),
                                 "api": int(traffic_counts.get("api", 0)),
                                 "backup": int(traffic_counts.get("backup", 0))})

study_false_table = pd.DataFrame(study_false_rows)
study_false_table
```

Out[66]:

	Режим	k	web	api	backup
0	Без учителя	3	1	3	0
1	Без учителя	5	0	2	0
2	Без учителя	10	0	0	0
3	Без учителя	20	0	0	0
4	Без учителя	40	0	0	4
5	Только норма	3	2	2	0
6	Только норма	5	1	2	0
7	Только норма	10	1	0	0
8	Только норма	20	1	0	0
9	Только норма	40	1	0	0

По полученным результатам $k = 10$ является лучшим обоснованным ближайших соседей.

Ограничения метода kNN

1. Зависимость результата детекции аномалий от метрики расстояния.

2. Необходимость выбора количества соседей.

Отсутствуют формализованные методы определения количества ближайших соседей. Приходится использовать достаточно затратные экспериментальные способы (кросс-валидация).

3. Неодинаковая плотность нормальных режимов.

Обычный kNN использует абсолютные расстояния. Это создаёт проблемы, если разные нормальные режимы имеют разную плотность.

Например:

- веб-трафик создаёт большую плотную группу;
- API-трафик образует другую плотную группу;

- резервное копирование выполняется редко и образует небольшую разреженную группу.

Для редкого резервного копирования расстояния до соседей могут быть больше, чем для веб-трафика. Это не означает, что резервное копирование является атакой. Оно просто относится к менее плотному нормальному режиму.

Один глобальный порог может:

- хорошо работать для плотной группы;
- создавать ложные тревоги в разреженной группе;
- пропускать аномалии внутри особенно плотной группы.

Возможные меры:

- строить отдельные модели для разных типов трафика;
- использовать отдельные пороги для разных режимов;
- кластеризовать нормальный трафик перед применением kNN;
- применять LOF, сравнивающий локальные плотности.

Именно неодинаковая плотность является одним из основных оснований для перехода от обычного kNN к LOF.

5. Маскирование групповых атак.

При обучении без учителя атакующие события могут становиться соседями друг друга.

Подобная проблема возникает для:

- массового сканирования;
- ботнет-трафика;
- автоматизированных попыток подбора пароля;
- повторяющейся эксфильтрации.

Возможные меры:

- обучать модель только на нормальных данных;
- очищать неразмеченную выборку;
- ограничивать окно, по которому строится модель;
- учитывать временные и групповые признаки.

6. Проблема высокой размерности.

При увеличении количества признаков возникает «проклятие размерности». В многомерном пространстве объекты становятся удалёнными друг от друга, а расстояния до ближайших и дальних соседей начинают различаться слабее.

Возможные меры:

- отбор информативных признаков;

- удаление сильно коррелирующих и дублирующих признаков;
- PCA или другие методы уменьшения размерности;
- построение моделей по отдельным группам признаков.

Однако уменьшение размерности может удалить слабые, но важные признаки атаки.

4. Local Outlier Factor (LOF)

Local Outlier Factor (LOF) — алгоритм обнаружения аномалий, основанный на концепции локальной плотности достижимости (local reachability density).

Объект считается аномальным, если плотность данных около него существенно ниже плотности около его соседей.

В отличие от обычного kNN, LOF использует не только расстояние до соседей:

- kNN отвечает на вопрос: «Насколько далеко объект расположен от ближайших соседей?»
- LOF отвечает на вопрос: «Насколько область около объекта разреженнее областей около его соседей?»

Метод может применяться в двух режимах:

- обучение без учителя;
- обучение только на нормальных данных.

Алгоритм метода

Шаг 1. Формирование пространства признаков.

Каждое событие представляется вектором:

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{ij}, \ldots, x_{im}),$$

где:

- $\mathbf{x}_i$ — i-е событие;
- x_{ij} — значение j-го признака у i-го события;
- m — количество признаков.

Категориальные признаки, например протокол или тип сервиса, должны быть предварительно закодированы либо использованы для разделения данных на однородные группы.

Шаг 2. Масштабирование признаков.

LOF непосредственно использует расстояния, поэтому чувствителен к масштабу признаков.

Без масштабирования признак с наибольшими числовыми значениями будет доминировать при вычислении расстояния.

Для стандартизации используется формула:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

где:

- x_{ij} — исходное значение j-го признака у i-го события;
- z_{ij}— стандартизированное значение;
- μ_j — среднее значение j-го признака в обучающей выборке;
- σ_j — стандартное отклонение j-го признака в обучающей выборке.

Различие режимов.

При обучении без учителя параметры масштабирования вычисляются по всей неразмеченной выборке. Недостаток заключается в том, что атаки также влияют на средние значения и стандартные отклонения.

При обучении только на нормальных данных параметры масштабирования вычисляются исключительно по нормальной обучающей выборке. Для новых событий используются уже рассчитанные параметры. Пересчитывать масштабирование по мониторинговой выборке нельзя.

Шаг 3. Выбор количества соседей.

Параметр k определяет, сколько ближайших объектов участвует в расчёте оценки аномальности:

$$1 \leq k < n,$$

где n — количество объектов в обучающей выборке.

Подходы к выбору количества ближайших соседей аналогичны методу kNN.

Шаг 4. Выбор метрики расстояния.

Метрика, так же как и количество ближайших соседей, является настраиваемым параметром. Для его выбора рекомендуется использовать экспериментальные подходы на пример описанного выше для выбора значения k.

Шаг 5. Вычисление расстояний.

Для каждой пары объектов вычисляется расстояние. Например, евклидово:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{m} (x_j - y_j)^2},$$

где:

- $\mathbf{x}$ — первый объект;
- $\mathbf{y}$ — второй объект;
- x_j — значение j-го признака объекта $\mathbf{x}$;
- y_j — значение j-го признака объекта $\mathbf{y}$;
- m — количество признаков.

Чем меньше расстояние, тем сильнее объекты похожи.

Шаг 6. Определение расстояния до k-го соседа.

Для объекта $\mathbf{x}$ расстояния до остальных объектов упорядочиваются:

$$d_1(\mathbf{x}) \leq d_2(\mathbf{x}) \leq \ldots \; d_q(\mathbf{x}) \leq \ldots \leq d_{n-1}(\mathbf{x}),$$

где:

- $d_q(\mathbf{x})$ — расстояние от $\mathbf{x}$ до q-го ближайшего соседа;
- $n - 1$ — количество остальных объектов;
- $\mathbf{x}$ — анализируемый объект;
- индексы показывают порядок после сортировки.

Расстояние до k-го соседа называется k-расстоянием:

$$k\text{-distance}(\mathbf{x}) = d_k(\mathbf{x}),$$

где:

- $k\text{-distance}(\mathbf{x})$ — расстояние от объекта $\mathbf{x}$ до его k-го ближайшего соседа;
- $d_k(\mathbf{x})$ — k-е расстояние в упорядоченном списке;
- k — число соседей;
- $\mathbf{x}$ — анализируемый объект.

Сам объект не должен считаться своим соседом.

Шаг 7. Формирование k-окрестности.

В k-окрестность входят все объекты, расстояние до которых не больше расстояния до k-го соседа:

$$N_k(\mathbf{x}) = \{\mathbf{q} \neq \mathbf{x} \mid d(\mathbf{x}, \mathbf{q}) \leq k\text{-distance}(\mathbf{x})\},$$

где:

- $N_k(\mathbf{x})$ — множество ближайших соседей объекта $\mathbf{x}$;
- $\mathbf{x}$ — анализируемый объект;
- $\mathbf{q}$ — q-й соседний объект;
- $\mathbf{q} \neq \mathbf{x}$ — объект не является соседом самого себя;
- $d(\mathbf{x}, \mathbf{q})$ — расстояние между объектами;
- $k\text{-distance}(\mathbf{x})$ — расстояние до k-го соседа.

Шаг 8. Вычисление достижимого расстояния.

LOF использует не обычное, а достижимое расстояние — reachability distance:

$$\text{reach-dist}_k(\mathbf{x}, \mathbf{q}) = \max\left\{k\text{-distance}(\mathbf{q}), d(\mathbf{x}, \mathbf{q})\right\},$$

где:

- $k\text{-distance}(\mathbf{q})$ — расстояние от соседа $\mathbf{q}$ до его собственного k-го соседа;
- $d(\mathbf{x}, \mathbf{q})$ — обычное расстояние между объектами $\mathbf{x}$ и $\mathbf{q}$.

Обратите внимание: используется k-расстояние соседа $\mathbf{q}$ а не анализируемого объекта $\mathbf{x}$.

Использование максимума делает оценку более устойчивой к очень близким точкам:

- если точка $\mathbf{q}$ находится в плотной области (то есть её $k\text{-distance}(\mathbf{q})$ невелико), но сама точка $\mathbf{x}$ от неё далеко, то в расчёт пойдёт реальное расстояние $d(\mathbf{x}, \mathbf{q})$;
- если точка $\mathbf{q}$ находится в разреженной области, то доминировать будет $k\text{-distance}(\mathbf{q})$, что сглаживает резкие локальные флуктуации.

Шаг 9. Вычисление локальной достижимой плотности (local reachability density - lrd).

Локальная достижимая плотность определяется по следующему выражению:

$$\text{lrd}_k(\mathbf{x}) = \frac{|N_k(\mathbf{x})|}{\sum\limits_{\mathbf{q} \in N_k(\mathbf{x})} \text{reach-dist}_k(\mathbf{x}, \mathbf{q})},$$

где:

- $N_k(\mathbf{x})$ — k-окрестность объекта;
- $|N_k(\mathbf{x})|$ — количество соседей;
- $\text{reach-dist}_k(\mathbf{x}, \mathbf{q})$ — достижимое расстояние.

В знаменателе — среднее расстояние достижимости от точки $\mathbf{x}$ до её соседей. Обратная величина — потому что нам нужна именно плотность: чем дальше соседи, тем реже точка, тем меньше плотность.

Шаг 10. Сравнение плотности объекта с плотностью соседей.

На этом шаге вычисляется локальный фактор выброса объекта - Local Outlier Factor (LOF):

$$\text{LOF}_k(\mathbf{x}) = \frac{1}{|N_k(\mathbf{x})|} \sum_{\mathbf{q} \in N_k(\mathbf{x})} \frac{\text{lrd}_k(\mathbf{q})}{\text{lrd}_k(\mathbf{x})},$$

где:

- k — количество соседей;
- $\mathbf{x}$ — анализируемый объект;
- $N_k(\mathbf{x})$ — множество его ближайших соседей;

- $|N_k(\mathbf{x})|$ — количество соседей;
- $\mathbf{q}$ — q-й сосед;
- $\mathrm{lrd}_k(\mathbf{q})$ — локальная достижимая плотность соседа;
- $\mathrm{lrd}_k(\mathbf{x})$ — локальная достижимая плотность анализируемого объекта.

Формулу можно представить как отношение:

$$\mathrm{LOF}_k(\mathbf{x}) = \frac{\text{средняя локальная плотность соседей}}{\text{локальная плотность объекта}}.$$

Интерпретация значений:

- $\mathrm{LOF} \approx 1$: плотность объекта примерно совпадает с плотностью его соседей — точка типична для своего локального окружения;
- $\mathrm{LOF} > 1$: объект находится в более разреженной области по сравнению с соседями — кандидат в аномалии;
- $\mathrm{LOF} \gg 1$: объект сильно выделяется на фоне локальной окрестности — сильная локальная аномалия;
- $\mathrm{LOF} < 1$: объект находится глубоко внутри плотного кластера (его плотность выше, чем у соседей). Обычно он не считается аномальным.

Универсального порога, например строго `LOF > 1`, нет. Из-за шума нормальные объекты также могут иметь значения немного выше единицы.

Шаг 11. Выбор порога принятия решения об анамальности.

Процедура выбора порога отличается для различных режимов обучения.

Выбор порога в режиме обучения без учителя.

1. Вычисление LOF для всей выборки.

Для каждого объекта неразмеченной выборки вычисляется:

$$L_i = \mathrm{LOF}_k(\mathbf{x}_i), \qquad i = 1, \ldots, n.$$

Для объекта $\mathbf{x}_i$ соседи выбираются без учета этого объекта.

2. Выбор порога.

Пусть предполагаемая доля аномалий равна α. Тогда порог можно определить как:

$$T_{\text{unsup}} = Q_{1-\alpha}(L_1, L_2, \ldots, L_n),$$

где $Q_{1-\alpha}$ — квантиль уровня $1 - \alpha$.

Например, при $\alpha = 0{,}05$ порог соответствует 95-му процентилю оценок.

3. Получение метки события.

$$\hat{y}_i = \begin{cases} 1, & L_i > T_{\text{unsup}}, \\ 0, & L_i \le T_{\text{unsup}}, \end{cases}$$

где:

- $\hat{y}_i$ — предсказанная метка события;
- $\hat{y}_i = 1$ — аномалия;
- $\hat{y}_i = 0$ — нормальное событие.

Особенность режима.

Атаки участвуют в формировании окрестностей и плотностей. Если атакующие объекты образуют достаточно плотную группу, они могут поддерживать локальную плотность друг друга.

Например, многочисленные похожие сканирования портов могут образовать собственную плотную область и получить LOF около единицы.

Выбор порога при обучении только на нормальных данных.

1. Разделение нормальных данных.

Нормальные данные разделяются на обучающую и калибровочную части:

$$X_{\text{normal}} = X_{\text{fit}} \cup X_{\text{calibration}},$$

$$X_{\text{fit}} \cap X_{\text{calibration}} = \varnothing,$$

где:

- X_{normal} — все доступные нормальные данные;
- X_{fit} — нормальные данные для построения окрестностей и плотностей;
- $X_{\text{calibration}}$ — нормальные данные для выбора порога.

2. Построение эталонной модели.

В качестве эталонного набора используется: X_{fit}.

Для каждого эталонного объекта рассчитываются:

- k-расстояние;
- множество соседей;
- достижимые расстояния;
- локальная достижимая плотность.

Атаки в этих расчётах не участвуют.

3. Оценивание калибровочных объектов.

Для каждого калибровочного нормального события вычисляется:

$$L_i^c = \mathrm{LOF}_k\left(\mathbf{x}_i^c \mid X_{\text{fit}}\right),$$

где:

- L_i^c — LOF-оценка i-го калибровочного объекта;
- $\mathbf{x}_i^c$ — i-й калибровочный нормальный объект;

- c — обозначение калибровочных данных;
- X_{fit} — эталонная нормальная выборка;
- k — количество соседей.

Калибровочные объекты не добавляются в эталонную выборку. Каждый из них рассматривается как новое событие.

4. Выбор порога.

Пусть допустимая доля ложных тревог равна α. Тогда:

$$T_{\text{normal}} = Q_{1-\alpha}\left(L_1^c, L_2^c, \ldots, L_m^c\right),$$

где:

- T_{normal} — порог, полученный по нормальным данным;
- $Q_{1-\alpha}$ — квантиль уровня $1-\alpha$;
- L_i^c — LOF-оценка калибровочного события;
- m — количество калибровочных объектов.

Шаг 12. Анализ нового события.

Для нового события $\mathbf{x}_{new}$ вычисляется:

$$L_{new} = \mathrm{LOF}_k(\mathbf{x}_{new} \mid X_{\text{fit}}),$$

где:

- L_{new} — LOF-оценка нового события;
- $\mathbf{x}_{new}$ — новое мониторинговое событие;
- X_{fit} — нормальная эталонная выборка;
- k — количество соседей.

Итоговая метка:

$$\hat{y}_{new} = \begin{cases} 1, & L_{new} > T_{\text{normal}}, \\ 0, & L_{new} \le T_{\text{normal}}. \end{cases}$$

Реализация LOF в sklearn

Для реализации метода LOF можно использовать класс `LocalOutlierFactor` модуля `neighbors` библиотеки `sklearn`.

Основные параметры:

- `n_neighbors` (по умолчанию 20): число соседей k, по которым считается локальная плотность;
- `metric` (по умолчанию `minkowski`): задаёт способ вычисления расстояния между точками. Наиболее распространенные: `euclidean`, `manhattan`, `chebyshev`, `minkowski`, `cosine`;

- `p` (по умолчанию 2): определяет вид метрики. Имеет смысл только когда `metric=minkowski`, в остальных случаях он игнорируется;
- `contamination` (по умолчанию `auto`): доля выбросов в данных. Влияет на то, как модель интерпретирует порог аномальности:
  - `contamination=auto`: порог для разделения на выбросы и нормальные точки алгоритм определяет сам, опираясь на подход из оригинальной статьи авторов LOF;
  - число с плавающей точкой (`float`): явно задаётся ожидаемая доля выбросов в данных. Например, при `contamination=0,05` примерно 5 % точек из обучающей выборки получат метку аномалии. Но есть важное ограничение: такое число должно лежать в диапазоне (0, 0.5]. Указывать 0 или значения больше 0.5 нельзя — это вызовет ошибку;
- `novelty` (по умолчанию `False`): меняет режим работы:
  - `novelty=False` (классический LOF): используется в режиме обучения без учителя (outlier detection), т.е. поиск выбросов в уже имеющейся смешанной выборке;
  - `novelty=True`: режим обучения только на нормальных данных (novelty detection). Модель обучается на «чистых» нормальных данных.

Реализация при обучении без учителя

В этом режиме вся неразмеченная мониторинговая выборка используется одновременно:

- для построения локальных окрестностей;
- для расчёта плотностей;
- для поиска аномалий.

In [35]:
```
monitoring_df[FEATURES].head()
```

Out[35]:

	connections_per_min	unique_destinations	outbound_mb
600	56.50	15.11	22.97
601	57.21	19.07	26.54
602	51.52	13.02	22.68
603	40.72	14.97	30.16
604	53.06	13.11	23.70

Будем использовать сформированную ранее стандартизованную мониторинговую выборку.

In [36]:
```
X_unsupervised_scaled[:5]
```

Out[36]:
```
array([[-0.47948747,  0.09505378, -0.35035616],
       [-0.46380287,  0.29426388, -0.30828921],
       [-0.58950058, -0.01008488, -0.35377337],
       [-0.82808324,  0.088011  , -0.26563308],
       [-0.55548046, -0.00555738, -0.34175424]])
```

Создание объекта LOF.

Выберем:

- `n_neighbors = 20`;
- евклидово расстояние;
- предполагаемую долю аномалий 5,5%.

In [37]:
```python
from sklearn.neighbors import LocalOutlierFactor

N_NEIGHBORS = 20

# Предполагаемая доля аномалий.
#
# Она не должна вычисляться по столбцу is_attack.
# В реальной системе это экспертная или эксплуатационная
# настройка, определяющая ожидаемый бюджет тревог.
EXPECTED_ANOMALY_FRACTION = 0.055

lof_unsupervised = LocalOutlierFactor(
    # Количество соседей для оценки локальной плотности.
    n_neighbors = N_NEIGHBORS,

    # Явно задаём евклидово расстояние.
    metric = "euclidean",

    # Доля объектов, которые должны быть отнесены
    # к аномалиям при выборе внутреннего порога.
    contamination = EXPECTED_ANOMALY_FRACTION,

    # Стандартный режим LOF:
    # поиск выбросов внутри обучающей выборки.
    novelty = False)
```

Обучение модели и формирование меток аномальности.

Для формирование меток объектов используется метод `fit_predict()`, который делает следующие действия в одной операции:

1. Обучает модель на переданных данных (для каждой точки вычисляются k ближайших соседей, расстояния достижимости, локальная достижимая плотность (LRD)).
2. Считает `negative_outlier_factor_` - это просто минус значение LOF для каждой точки:

$$\texttt{negative_outlier_factor_}[i] = -\,\mathrm{LOF}(x_i).$$

В `LocalOutlierFactor` из `scikit-learn` решили хранить `negative_outlier_factor_`, чтобы согласовать логику с другими детекторами аномалий, где «чем меньше оценка — тем более аномально».

3. Сортирует значения `negative_outlier_factor_` по возрастанию.
4. На основе параметра `contamination` подбирает порог (`offset_`), чтобы доля точек с меткой -1 была примерно равна `contamination`.
5. Возвращает метки: 1 — норма, (-1) — аномалия. Точки с `negative_outlier_factor_ < offset_` помечаются как аномалии (-1).

In [38]:
```
# Получение меток
lof_unsupervised_sklearn_labels = (
    lof_unsupervised.fit_predict(X_unsupervised_scaled))
```

Получение положительной оценки аномальности `negative_outlier_factor_`.

In [39]:
```
# negative_outlier_factor_:
# значения около -1 соответствуют нормальным объектам;
# более отрицательные значения — более аномальным.
negative_lof_unsupervised = (lof_unsupervised.negative_outlier_factor_)

# Меняем знак:
# теперь большие значения соответствуют аномалиям.
lof_unsupervised_scores = (-negative_lof_unsupervised)

print(f'Первые оценки negative_outlier_factor_: {negative_lof_unsupervised[:5]}'
print(f'Первые положительные оценки аномальности: {lof_unsupervised_scores[:5]}'
```

```
Первые оценки negative_outlier_factor_: [-0.97439776 -1.11023031 -0.96848103 -1.2
8029713 -0.95255173]
Первые положительные оценки аномальности: [0.97439776 1.11023031 0.96848103 1.280
29713 0.95255173]
```

Внутренний порог `offset_`.

In [40]:
```
print(f'Внутренний порог аномальности: {lof_unsupervised.offset_}')
```

```
Внутренний порог аномальности: -2.369265064019858
```

В исходной системе знаков объект считается аномальным, если:

$$\text{negative_outlier_factor} < \text{offset}.$$

После изменения знака:

$$A(x) > -\text{offset}.$$

Поэтому положительный порог равен:

In [41]:
```
lof_unsupervised_threshold = (-lof_unsupervised.offset_)
print(f'Положительный порог аномальности: {lof_unsupervised_threshold}')
```

```
Положительный порог аномальности: 2.369265064019858
```

Преобразование предсказаний.

Преобразуем метки LOF:

- `skearn`:
  - 1 — нормальный объект;

    - -1 — аномалия;
- в наборе данных:
    - 0 — нормальный объект;
    - 1 — аномалия.

In [42]:
```
lof_unsupervised_prediction = (lof_unsupervised_sklearn_labels== -1).astype(int)
lof_unsupervised_prediction
```

Out[42]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

Сохранение результатов.

In [43]:
```
monitoring_df["lof_unsupervised_score"] = (lof_unsupervised_scores)
monitoring_df["lof_unsupervised_anomaly"] = (lof_unsupervised_prediction)

monitoring_df[FEATURES + ['lof_unsupervised_score'] + ['lof_unsupervised_anomaly
```

Out[43]:

	connections_per_min	unique_destinations	outbound_mb	lof_unsupervised_score	l
837	16.96	2.25	177.34	1.156712	
838	19.73	2.20	216.89	0.988245	
839	16.14	0.91	184.11	1.079991	
840	218.28	167.82	4.79	20.669105	
841	205.60	168.93	5.68	21.895332	
842	219.44	146.29	6.08	17.468434	
843	215.66	161.48	5.54	19.993343	
844	55.73	13.29	644.75	9.722077	
845	58.14	11.27	601.34	8.901191	
846	56.84	14.26	664.41	10.108926	
847	76.22	0.94	3.39	3.762231	
848	74.86	0.90	2.84	3.709201	
849	81.06	0.65	3.25	3.871453	
850	69.96	0.98	2.78	3.795931	
851	134.57	46.17	18.86	6.597422	
852	134.50	47.52	24.31	6.712742	
853	152.45	46.08	20.14	6.932768	

Оценивание качества работы метода LOF.

In [44]:
```
print_detection_result(method_name = "LOF: обучение без учителя",
                       y_true = monitoring_df["is_attack"],
                       y_pred = monitoring_df["lof_unsupervised_anomaly"])
```

```
LOF: обучение без учителя
TP — обнаруженные атаки: 14
FP — ложные тревоги: 0
FN — пропущенные атаки: 0
TN — корректно распознанная норма: 240
```

Интерпретация результатов выявления аномалий методом LOF в задачах кибербезопасности.

1. Не использовать метки, полученные в методе `fit_predict()` как истину. Параметр `contamination` лишь гарантирует долю аномалий (-1), а не то, что все они - реальные атаки.
2. Нужно смотреть на `negative_outlier_factor_` как на скоринг-оценку. По ней удобно ранжировать инциденты и отдавать на ручной разбор события с оценками из заданного топ-N.
3. Использовать метки 1/-1, когда:

- нужно быстро получить «список кандидатов» под заданный бюджет разбора (например, «дай 5 % самых подозрительных событий, чтобы аналитик посмотрел вручную»);
- для создания простой демонстрации: «вот эти точки модель считает аномалиями»;
- сравнения метрик качества модели.

4. Использовать `negative_outlier_factor_` (или пересчитанный LOF), когда нужно ранжировать инциденты: не просто «аномалия/не аномалия», а «самые подозрительные → менее подозрительные». Это критично для SOC: сначала смотрят топ-20, а не все 5 %.

Визуализация работы метода LOF.

In [57]:
```
# @title
def plot_lof_scores(
    data,
    score_column,
    threshold,
    title
):
    """
    Строит распределение положительных LOF-оценок.

    Истинные метки используются только для учебного
    анализа качества.
    """

    normal_scores = data.loc[
        data["is_attack"] == 0,
        score_column
    ]

    attack_scores = data.loc[
        data["is_attack"] == 1,
        score_column
    ]

    fig, ax = plt.subplots(figsize=(6, 4))

    ax.hist(
        normal_scores,
        bins=30,
        alpha=0.65,
        color="steelblue",
        label="Нормальные события"
    )

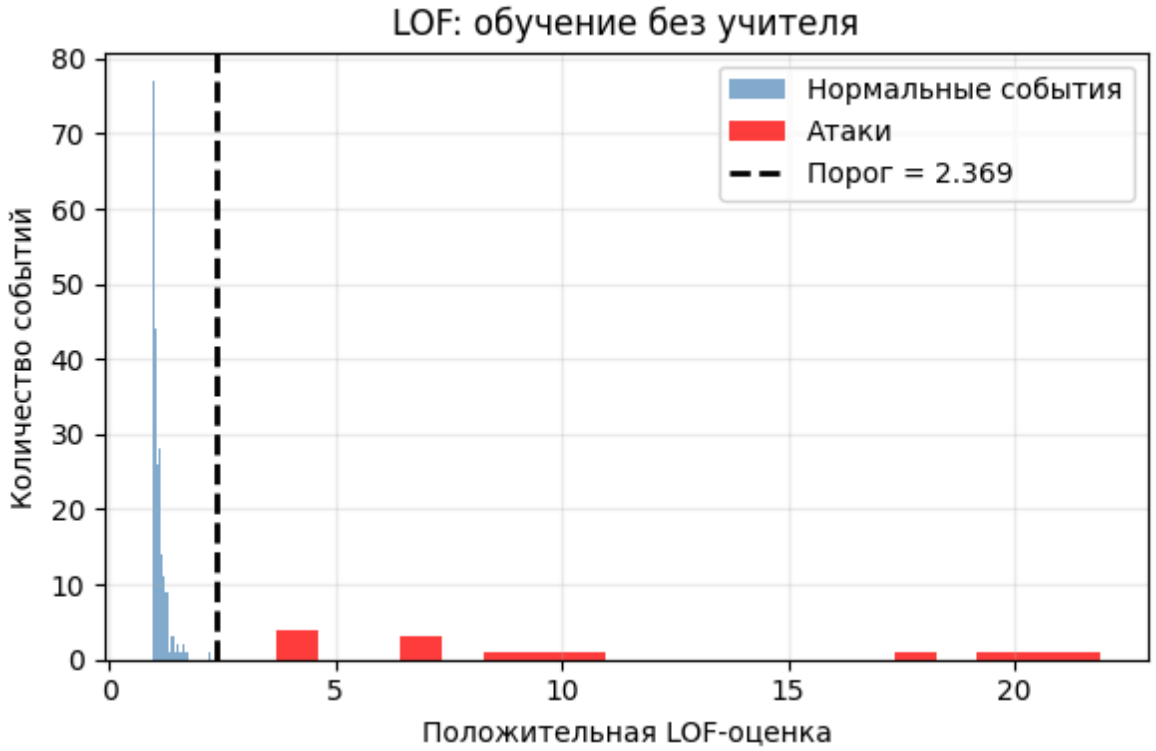
    ax.hist(
        attack_scores,
        bins=20,
        alpha=0.75,
        color="red",
        label="Атаки"
    )
```

```
    ax.axvline(
        threshold,
        color="black",
        linestyle="--",
        linewidth=2,
        label=f"Порог = {threshold:.3f}"
    )

    ax.set_title(title)
    ax.set_xlabel("Положительная LOF-оценка")
    ax.set_ylabel("Количество событий")
    ax.grid(alpha=0.25)
    ax.legend()

    plt.tight_layout()
    plt.show()
```

In [58]:
```
plot_lof_scores(data = monitoring_df,
                score_column = "lof_unsupervised_score",
                threshold = lof_unsupervised_threshold,
                title = "LOF: обучение без учителя")
```



Реализация при обучении только на нормальных данных

В этом режиме LOF строится только по подтверждённым нормальным событиям.

Затем новые события проверяются относительно нормального эталона.

В терминологии `sklearn` это **novelty detection**, поэтому необходимо установить: `novelty=True`.

Разделение эталонной нормы.

Нормальные данные разделяются на:

- `normal_fit` — построение локальных окрестностей;
- `normal_calibration` — выбор порога.

`reference_df` — датафрейм, который содержит только нормальные наблюдения. Будет использоваться для обучения модели.

`monitoring_df` — датафрейм, который содержит как нормальные наблюдения, так и атаки. Будет использоваться для проверки новых событий.

In [48]:
```
normal_fit_df, normal_calibration_df = train_test_split(
    reference_df,

    # 25% нормальных событий оставляем
    # для калибровки порога.
    test_size = 0.25,
    random_state=42,

    # Сохраняем пропорции нормальных режимов:
    # web, api и backup.
    stratify = reference_df["traffic_type"])

# Сбрасываем индексы, чтобы позиции объектов,
# возвращаемые kneighbors(), совпадали с iloc.
normal_fit_df = (normal_fit_df.reset_index(drop = True))
normal_calibration_df = (normal_calibration_df.reset_index(drop = True))

print("Норма для обучения:", len(normal_fit_df))
print("Норма для калибровки:", len(normal_calibration_df))
```

```
Норма для обучения: 450
Норма для калибровки: 150
```

Масштабирование признаков.

Для масштабирования признаков используется только обучающая нормальная выборка.

In [49]:
```
X_normal_fit = (normal_fit_df[FEATURES].to_numpy())
X_normal_calibration = (normal_calibration_df[FEATURES].to_numpy())
X_monitoring = (monitoring_df[FEATURES].to_numpy())

# Масштабирование обучается только по normal_fit.
scaler_normal = StandardScaler()

X_normal_fit_scaled = (scaler_normal.fit_transform(X_normal_fit))

# Калибровочные и мониторинговые данные
# преобразуются тем же scaler.
# fit_transform() для них применять нельзя.
X_normal_calibration_scaled = (scaler_normal.transform(X_normal_calibration))
X_monitoring_scaled = (scaler_normal.transform(X_monitoring))
```

Создание LOF с `novelty=True`.

Важное отличие от режима обучения без учителя: в режиме `novelty=True` нет метода `fit_predict`. Доступны два отдельных метода: `fit()` и `predict()`.

Метод `fit()` строит профиль «нормы» на переданных данных и фиксирует порог для отделения аномалий. Его цель — не найти выбросы в этой выборке, а «запомнить», как выглядит нормальное поведение, чтобы потом детектировать отклонения на новых данных:

- вычисление достижимых расстояний и локальной достижимой плотности (LRD) для каждой обучающей точки;
- вычисление LOF для обучающих точек и переход к отрицательным оценкам:

$$\texttt{negative_outlier_factor_}[i] = -\,\mathrm{LOF}(x_i).$$

Этот атрибут сохраняется и нужен для анализа качества профиля нормы и диагностики (например, чтобы убедиться, что в «нормальных» данных нет явных выбросов).

- подбор порога `offset_` на основе значения параметра `contamination`. Этот порог позже будет использоваться в методе `predict()`;
- сохранение внутренних параметров.

В объекте сохраняются:

- структура соседей (индексы),
- `negative_outlier_factor_` для обучающих данных,
- `offset_` (фиксированный порог),
- параметры модели (`n_neighbors`, `metric` и т. д.).

В этом режиме параметр `contamination` — это доля «пограничных» нормальных точек, которые модель формально пометит как аномалии, чтобы подобрать порог (`offset_`). Это не относится к реальным атакам (их в выборке нет), а определяет насколько строгим будет порог относительно профиля нормы.

In [50]:
```
lof_normal = LocalOutlierFactor(n_neighbors = N_NEIGHBORS,
                                metric = "euclidean",

    # Внутренний порог модели использовать не будем.
    # Порог будет рассчитан вручную по отдельной
    # нормальной калибровочной выборке.
    contamination = "auto",

    # Разрешаем оценивать новые, ранее не встречавшиеся события.
    novelty = True)

# fit() получает только нормальные обучающие события.
lof_normal.fit(X_normal_fit_scaled)
```

Out[50]:

LocalOutlierFactor

```
LocalOutlierFactor(metric='euclidean', novelty=True)
```

Получение оценок аномальности объектов калибровочной выборки.

При `novelty=True` для формирования непрерывных оценок аномальности используется метод `score_samples(X_new)`, который выполняет следующие действия:

- для каждой новой точки из множества X_{new} находит k ближайших соседей среди обучающих данных (X_{normal});
- считает достижимые расстояния и локальную достижимую плотность (LRD) — точно так же, как в оригинальном LOF, но соседи берутся только из обучающей выборки (модель не "смотрит" на другие точки из X_{new}, а только на выученный профиль нормы);
- вычисляет LOF для каждой новой точки;
- переводит LOF в "сырую" оценку аномальности. Это делается как `score = -LOF` (аналогично атрибуту `negative_outlier_factor_`, но для новых данных).

In [51]:
```
# score_samples() возвращает противоположную LOF-оценку:
# большее значение – объект нормальнее;
# более отрицательное – объект аномальнее.
calibration_normality_scores = (lof_normal.score_samples(X_normal_calibration_sc

# Меняем знак, чтобы получить привычную оценку:
# большее значение – объект аномальнее.
lof_calibration_scores = (-calibration_normality_scores)

print(f'Первые исходные оценки score_samples: {calibration_normality_scores[:5]}
print(f'Первые положительные оценки аномальности: {lof_calibration_scores[:5]}')
```

```
Первые исходные оценки score_samples: [-1.08327283 -0.99157869 -2.59029793 -0.957
39816 -1.07957242]
Первые положительные оценки аномальности: [1.08327283 0.99157869 2.59029793 0.957
39816 1.07957242]
```

Ручной выбор порога аномальности.

Выберем 99-й процентиль оценок нормальных калибровочных событий.

Около 99% новых нормальных калибровочных событий имеют LOF-оценку не выше выбранного значения.

Целевая частота ложных тревог составляет приблизительно 1%, если будущий нормальный трафик соответствует калибровочным данным.

In [52]:
```
NORMAL_SCORE_QUANTILE = 0.99

lof_normal_threshold = np.quantile(lof_calibration_scores,
                                   NORMAL_SCORE_QUANTILE)

print(f'Порог LOF при обучении только на норме: {lof_normal_threshold}')
```

```
Порог LOF при обучении только на норме: 1.8006445377926636
```

Оценка мониторинговых событий.

In [53]:
```
# Мониторинговые события не участвовали в fit(),
# поэтому их можно передать в score_samples().
monitoring_normality_scores = (lof_normal.score_samples(X_monitoring_scaled))

# Преобразуем оценки в формат:
# большее значение — сильнее аномальность.
lof_normal_scores = (-monitoring_normality_scores)

# Ручное сравнение с калибровочным порогом.
lof_normal_prediction = (lof_normal_scores > lof_normal_threshold).astype(int)
```

Сохраним результаты.

In [54]:
```
monitoring_df["lof_normal_score"] = (lof_normal_scores)
monitoring_df["lof_normal_anomaly"] = (lof_normal_prediction)

monitoring_df[FEATURES + ['lof_normal_score'] + ['lof_normal_anomaly']].tail(17)
```

Out[54]:

	connections_per_min	unique_destinations	outbound_mb	lof_normal_score	lof_nor
837	16.96	2.25	177.34	1.562801	
838	19.73	2.20	216.89	0.966726	
839	16.14	0.91	184.11	1.449503	
840	218.28	167.82	4.79	82.045365	
841	205.60	168.93	5.68	82.547005	
842	219.44	146.29	6.08	70.353880	
843	215.66	161.48	5.54	78.572587	
844	55.73	13.29	644.75	16.937676	
845	58.14	11.27	601.34	15.033914	
846	56.84	14.26	664.41	17.828878	
847	76.22	0.94	3.39	2.911755	
848	74.86	0.90	2.84	2.967646	
849	81.06	0.65	3.25	3.130101	
850	69.96	0.98	2.78	3.027660	
851	134.57	46.17	18.86	15.389013	
852	134.50	47.52	24.31	16.087532	
853	152.45	46.08	20.14	15.679719	

Важное замечание.

Технически при `novelty=True` для оценивания мониторинговых объектов можно было использовать метод `predict()`, который выполняет те же действия, что и метод `score_samples()` плюс дополнительно:

- применяет порог `offset_`, который был зафиксирован на этапе обучения (метод `fit()`):
- если `score < offset_` → метка (-1) (аномалия/новизна), иначе → метка 1 (норма);
- возвращает массив меток длины `len(X_new)` со значениями 1 и -1.

Эти метки будут рассчитаны с использованием внутреннего `offset_`, связанного с параметром `contamination`.

В рассматриваемой реализации требуется порог по отдельным нормальным калибровочным данным. Поэтому используются `score_samples()` и ручное сравнение:

`lof_normal_scores > lof_normal_threshold`.

Не следует смешивать встроенные метки `predict()` и ручной калибровочный порог: это два разных правила принятия решения.

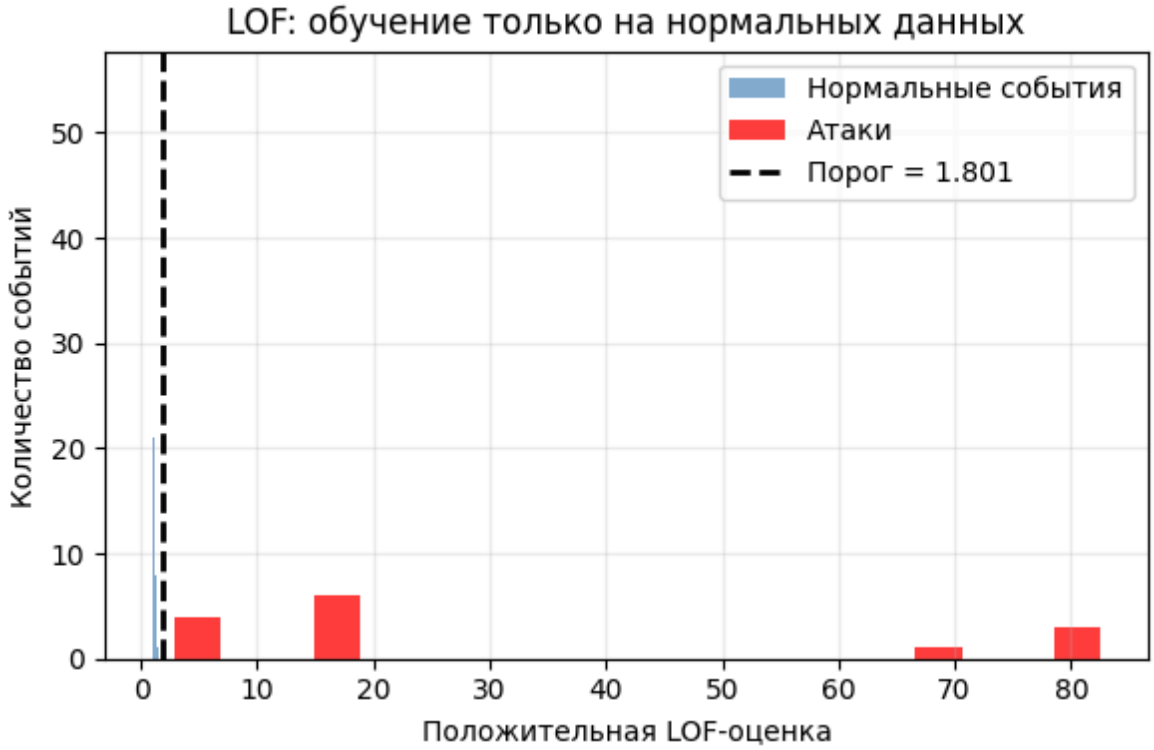
Оценивание качества детекции аномалий.

In [55]:
```
print_detection_result(method_name = ("LOF: обучение только на нормальных данных
                       y_true = monitoring_df["is_attack"],
                       y_pred = monitoring_df["lof_normal_anomaly"])
```

```
LOF: обучение только на нормальных данных
TP — обнаруженные атаки: 14
FP — ложные тревоги: 1
FN — пропущенные атаки: 0
TN — корректно распознанная норма: 239
```

Визуализация результатов метода LOF.

In [59]:
```
plot_lof_scores(data = monitoring_df,
                score_column = "lof_normal_score",
                threshold = lof_normal_threshold,
                title = "LOF: обучение только на нормальных данных")
```



Ограничения метода LOF

1. Чувствительность к выбору параметра `n_neighbors` (количество ближайших соседей).

Число соседей напрямую определяет, что будет считаться «локальной плотностью». При малом значении метод становится чувствителен к шуму: отдельные случайные выбросы могут сильно исказить оценку плотности. При большом - теряется локальность: LOF начинает усреднять по слишком широкой окрестности и пропускает «тонкие» локальные аномалии.

Например, короткая серия сканирующих пакетов может быть заметна при k=10, но «раствориться» при k=50. На практике рекомендуется перебрать несколько значений и выбрать то, которое обеспечивает требуемое качество выявления аномалий.

2. Проклятие размерности (curse of dimensionality).

В пространствах высокой размерности расстояния между точками становятся малоразличимыми: все пары точек оказываются примерно на одинаковом расстоянии. Из-за этого понятие «ближайших соседей» теряет смысл, а достижимые расстояния и оценки локальной плотности становятся нестабильными.

Если подавать на вход много признаков (например, десятки метрик на сессию, включая энтропии, доли, агрегированные статистики), LOF может работать плохо. Решения:

- снижение размерности (PCA, UMAP) перед LOF;

- отбор признаков под конкретную гипотезу атаки (например, для сканирования портов важнее доля уникальных dst_port и распределение длительности, а не все метрики сразу);
- комбинирование: LOF на небольшом наборе «плотных» признаков + отдельные правила/метрики для остальных.

3. Отсутствие универсального порога для LOF.

Значения LOF не имеют естественной вероятностной интерпретации. Нет формулы «LOF > 1.5 = аномалия», которая работала бы везде. В одних датасетах аномалии начинаются с LOF=2, в других даже LOF=3 может быть нормальным из-за сильной локальной неоднородности.

Практические подходы:

- подбор порога по историческим данным: посмотреть распределение LOF на заведомо «чистых» периодах и выбрать квантиль (например, 99-й перцентиль);
- динамический порог: в каждом временном окне считать текущий перцентиль LOF и помечать как аномальные точки выше него.

4. Нестабильность на разреженных и сильно неоднородных данных.

Если данные содержат области с очень разной плотностью (например, периоды с малым трафиком и периоды пиковой нагрузки), то «нормальные» точки в разреженной области могут получить высокий LOF просто потому, что у них мало соседей и локальная плотность объективно низкая.

Это частая ситуация в кибербезопасности: ночью трафик редкий и «разреженный», днём плотный. Если посчитать LOF по всему дню вместе, ночные точки могут ложно считаться аномалиями.

Решения:

- сегментация по режимам работы: отдельно моделировать «ночь» и «день» (или по типам трафика, по протоколам);
- временные окна: считать LOF внутри скользящих окон, чтобы сравнивать точки только с близкими по времени соседями.

5. Высокая вычислительная сложность.

Для каждой точки нужно найти заданное количество ближайших соседей. В наивной реализации сложность алгоритма $O(n^2)$. При анализе миллионов сессий прямой LOF на всём датасете может быть нереалистичным.

6. Интерпретируемость и объяснение решений.

LOF даёт числовую оценку аномальности, но не объясняет, какие именно признаки сделали точку аномальной. Это усложняет расследование инцидентов.

Чтобы компенсировать это, используют: -анализ соседей: смотрят, какие точки считаются соседями и чем аномальная точка от них отличается;

- локальные контрасты: сравнить значения ключевых признаков аномальной точки со средним/медианой её соседей;
- комбинированные отчёты: рядом с LOF показывать простые метрики (доля уникальных портов, энтропия, всплеск по bytes), которые легче объяснить SOC-аналитику.

Литература

1. Чио К., Фримэн Д. Машинное обучение и безопасность [Электронный ресурс]:. - Москва: ДМК Пресс, 2020. - 388 с. – Режим доступа: https://e.lanbook.com/book/131707.
2. Протодьяконов А. В., Пылов П. А., Садовников В. Е. Алгоритмы Data Science и их практическая реализация на Python [Электронный ресурс]:учебное пособие. - Вологда: Инфра-Инженерия, 2022. - 392 с. – Режим доступа: https://e.lanbook.com/book/281189.
3. Криволапов С.Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python [Электронный ресурс]:Учебное пособие. - Москва: ООО "Научно-издательский центр ИНФРА-М", 2025. - 678 с. – Режим доступа: https://znanium.ru/catalog/document?id=450872.
4. Коротеев М.В. Основы машинного обучения на Python [Электронный ресурс]:Учебник. - Москва: КноРус, 2025. - 431 с. – Режим доступа: https://book.ru/book/957785.
5. Груздев А. В. План, примеры и метрики качества [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 814 с. – Режим доступа: https://e.lanbook.com/book/314948.
6. Груздев А. В. Инструменты и валидация [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 816 с. – Режим доступа: https://e.lanbook.com/book/314945.